TEB: The Timed-Elastic-Band Planner, Start to Finish

Contents

1. Overview . . . 2
2. Where TEB sits: inputs, outputs, parameters . . . 5
   - 2.1 A controller plugin . . . 5
   - 2.2 The inputs on the room's numbers . . . 6
   - 2.3 The parameters, and the six we change . . . 7
3. Step 1: the plan ahead and the local goal . . . 9
4. Step 2: obstacles from the costmap . . . 11
   - 4.1 Cells become point obstacles . . . 11
   - 4.2 The footprint model and which obstacles each pose reads . . . 12
5. Step 3: warm start, or start again . . . 14
6. Step 4: candidates in different homotopy classes . . . 17
   - 6.1 Telling the sides apart: the H-signature . . . 17
   - 6.2 Finding new classes: a random roadmap . . . 20
7. Step 5: optimise every band: the g2o graph . . . 22
8. Step 6: pick the cheapest band . . . 25
9. Step 7: check, command and failure . . . 27
10. The whole drive . . . 29
11. Car-like robots . . . 31
12. DWA, TEB and MPC compared . . . 34
13. Summary . . . 37
14. Sources . . . 38
15. Key terms . . . 39

Where this fits:

- **Builds on:** Hybrid A*; Global plan and local planner; Local minima and homotopy classes; Timed elastic band (TEB); Trajectory optimisation; DWB controller, start to finish.

1. Overview

Key point: Every cycle, `teb_local_planner` cuts the global plan at the edge of its costmap, turns every occupied cell into an obstacle point, keeps up to 5 timed elastic bands that pass obstacles on different sides, optimises each with 4 passes of 5 Levenberg–Marquardt steps, picks the cheapest, checks its first poses and sends the speed of its first piece. On the room, our Python version of that algorithm passes the person 0.137 m clear and stops on the door pose at 9.8 s.

The robot can already avoid a person the map does not contain. RO-058's local planner (G-2911), a timed elastic band (G-2920), solves an optimisation problem every 0.1 s and stops on the door pose at 8.0 s, against 6.7 s in a clear room. Nav2's DWB controller, in DWA and DWB, stops in front of the person when the plan is made once, and is accepted at the door at 13.3 s when the plan is remade every second.

RO-058's band, however, was a teaching version of TEB, not the package a robot runs:

- it measured distances to the exact shapes of the table and the person, while the package receives only the cells of a costmap;
- it limited each wheel to 0.6 m/s, while the package has one limit for speed and one for turn rate;
- it kept one band, so with the person 0.25 m further left its warm start entered the gap between person and table and overlapped the person by 0.02 m (local minima);
- it never said which pose the package aims at, which of several bands it sends, or what it does when a band is unsafe.

This Note takes the real package, `teb_local_planner`, start to finish: what goes in, every step in the order its code runs them, and what comes out. The problem of one cycle, on numbers: at 2.4 s the robot drives at 0.5 m/s, a person it has never seen stands on its plan, and within the next 0.1 s TEB must turn a 3.698 m plan and 590 occupied costmap cells into one forward speed and one turn rate.

What we ran. The runs in this Note are our own Python version of `teb_local_planner`'s algorithm (`images/tebkit.py`), written from its code on the branch `ros2-master`, the one Nav2 uses, with its parameter values. Our version is not the package running: it follows the same steps in the same order, and we simplified four things:

- the solver: g2o's sparse Levenberg–Marquardt is replaced by our own, with the same cost terms, the same 4 passes of 5 steps and g2o's rule for the step size, but with derivatives by finite differences and no sparse linear algebra;
- the costmap: a cell is occupied when a shape of the room covers it, with no sensor; the safety check tests every occupied cell inside the robot's disk, while Nav2's tests the cells

under the disk's outline;

- the roadmap's random points come from a generator with a fixed seed, so every run repeats;
- planning takes no time: each command is applied at once for the next 0.1 s.

Figure 1 plays the whole drive. Watch the orange band, the one TEB picks each cycle, bend left in the cycle the person appears while a second, purple band is weighed against it, and the robot pass and turn up to the door.

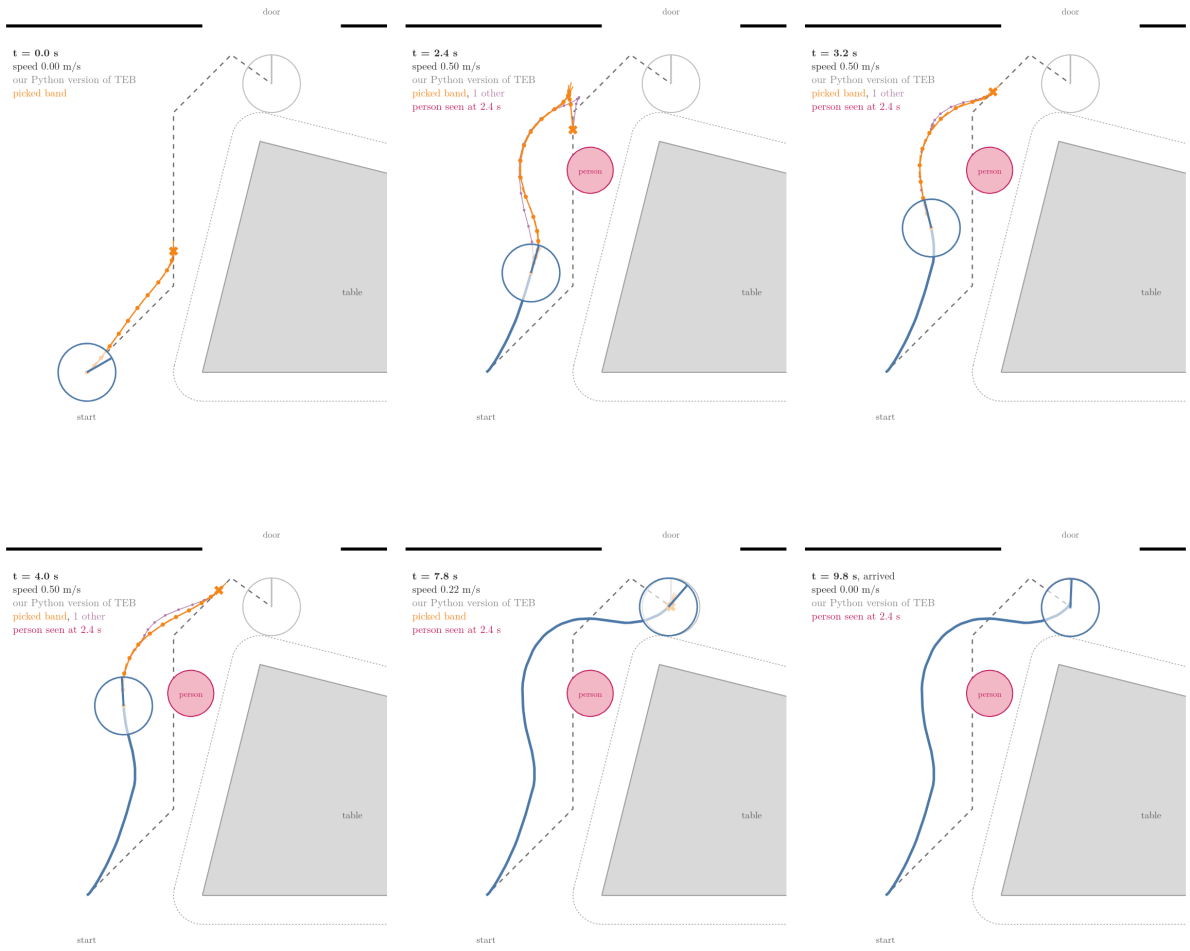


Figure 1: Our Python version of teb_local_planner's algorithm (images/tebkit.py, written from the ros2-master code, not the package running) drives the room, one frame per 0.2 s. Grey dashed: the global plan, RO-034's A* route with its ends moved to the robot and the door pose, made once. Orange: the band TEB picks in that cycle, its poses with headings; pale purple: the other candidate band, when there is one; orange cross: the local goal. At 2.4 s the person (pink) steps in. The last frame holds the arrival at 9.8 s: within 0.02 m and 0.05 rad of the door pose, stopped

The robot passes the person 0.137 m clear at 4.2 s, comes within 0.02 m of the door pose at 7.9 s still facing 35.5° off, and stops on it, turned, at 9.8 s (notebook). Figure 2 shows the seven steps of one cycle; Sections 3 to 9 take them in that order:

- what goes in and what comes out, and the parameters (Section 2);
- the plan ahead and the local goal (Section 3);
- obstacles from the costmap (Section 4);
- the warm start, or starting again (Section 5);
- candidate bands on different sides of obstacles (Section 6);
- optimising every band (Section 7) and picking one (Section 8);
- the safety check, the command, and what happens on failure (Section 9);
- the whole drive compared with RO-058 and DWB (Section 10), car-like robots (Section 11), and DWA, TEB and MPC compared (Section 12).

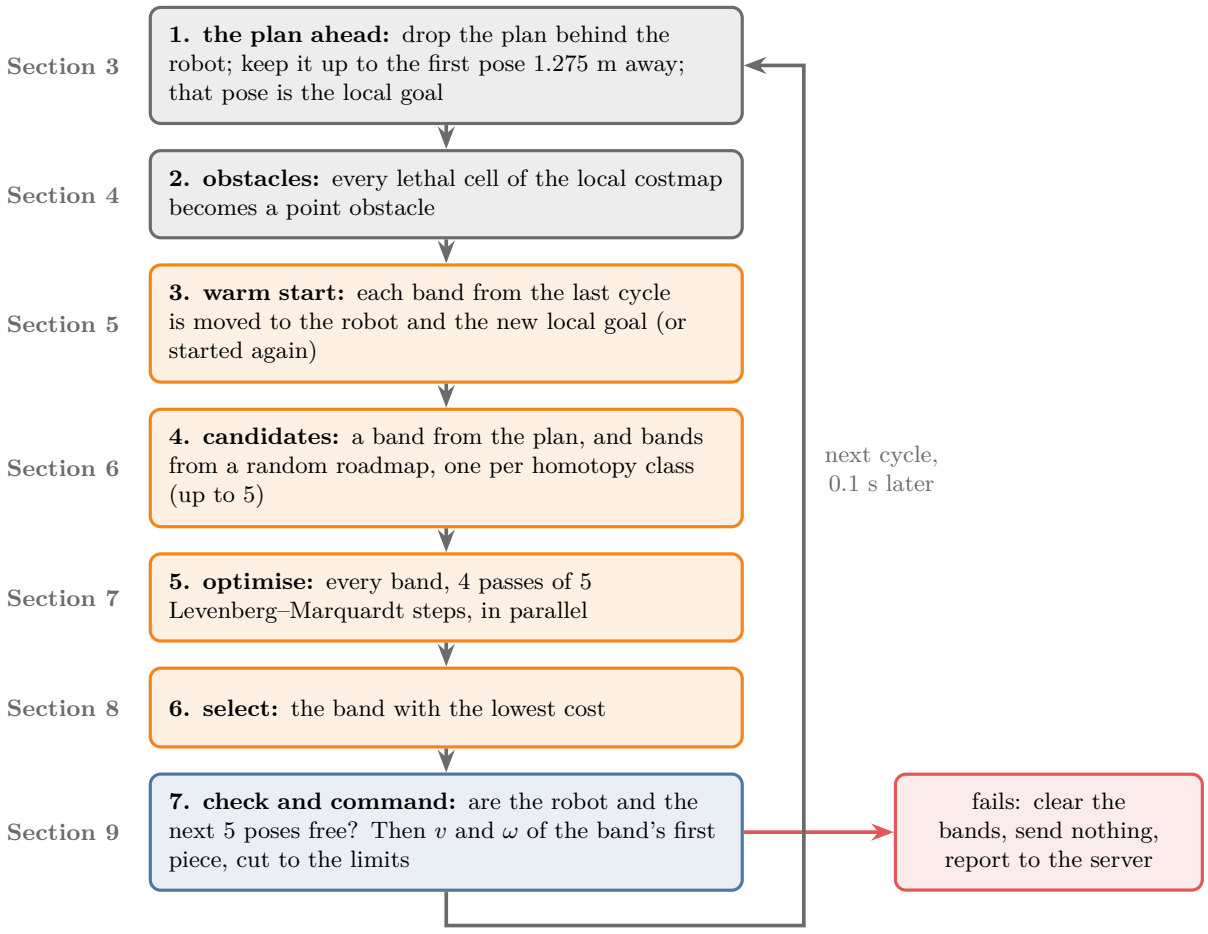


Figure 2: One cycle of teb_local_planner, in the order computeVelocityCommands() and HomotopyClassPlanner::plan() run it (ros2-master). Grey: the inputs are prepared; orange: the bands are made, optimised and compared; blue: the result is checked and sent. A failure clears the bands and reports to Nav2's controller server. The cycle repeats every 0.1 s here

2. Where TEB sits: inputs, outputs, parameters

> **Key point:** In Nav2, TEB is a controller plugin from its own repository. Each cycle the controller server passes it the robot's pose and speed; it reads the global plan it was given and the local costmap, and returns one forward speed and one turn rate, or an error. Its parameters default to the values in `teb_config.h`; we change six for our robot and room.

The opening's problem starts with the inputs: before TEB can turn a plan and some cells into a command, the controller server must give it the plan, the cells and the robot's state, every 0.1 s.

2.1 A controller plugin

The idea. Nav2 runs one program, the controller server, that calls a controller every cycle; which controller is a setting. A piece of code that a program loads by name from a setting, instead of having it built in, is a plugin. TEB is such a plugin: the package's example file for Nav2 loads `teb_local_planner::TebLocalPlannerROS` under the name `FollowPath` (teb_local_planner, `params/teb_params.yaml`). TEB is not part of Nav2's own repository: Nav2's list of plugins names it as a third-party controller and links to `rst-tu-dortmund/teb_local_planner` (Nav2 docs, navigation plugins). In ROS 1 the same planner is a plugin of `move_base` (teb_local_planner, `noetic-devel`). Nav2 start to finish follows one goal through every Nav2 server.

The controller server calls TEB three ways (teb_local_planner, `src/teb_local_planner_ros.cpp`):

1. `configure()`, once: read the parameters and build the planner, with several bands if `enable_homotopy_class_planning` is true (the default), or one;
2. `setPlan()`, whenever the global planner sends a new plan: TEB stores it and does not start again;
3. `computeVelocityCommands()`, every cycle: the robot's pose and speed go in; a speed and a turn rate come out, or an error.

Figure 3 shows what flows in and out.

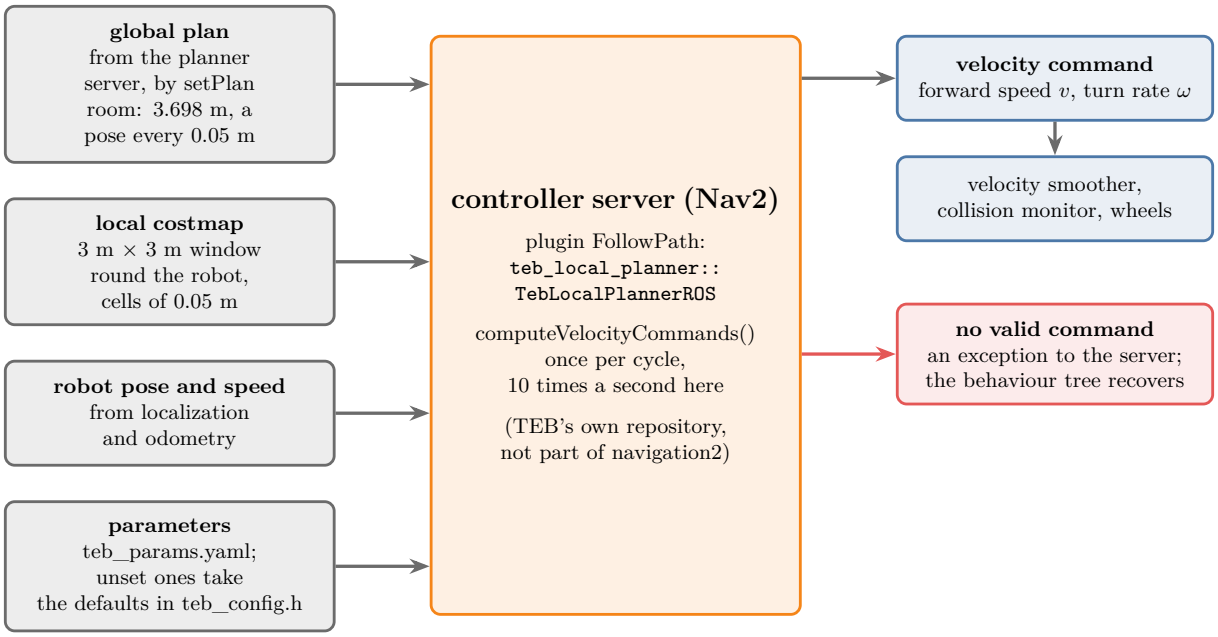


Figure 3: Where teb_local_planner sits in Nav2. Inputs, left: the global plan from the planner server (3.698 m, a pose about every 0.05 m in the room), the local costmap (3 m by 3 m round the robot, 0.05 m cells), the robot's pose and speed, and the parameters. The controller server calls computeVelocityCommands() each cycle. Outputs, right: a forward speed and a turn rate, which pass a velocity smoother and a collision monitor to the wheels; or, when no command can be made, an exception the server reports, and the behaviour tree recovers

2.2 The inputs on the room's numbers

What TEB gets in the room, in the cycle at 2.4 s and every other:

- **the global plan:** RO-034's A* route on 0.5 m cells, 3.414 m between cell centres, with its first point moved to the robot and its last to the door pose, as RO-058's refined drive did and as Nav2's planners do: 75 poses, 3.698 m, a pose about every 0.05 m, one per cell of a 0.05 m grid. As in RO-058, the plan is made once. In Nav2, the default behaviour tree of the released versions (Humble, Jazzy, Kilted) remakes it once a second, while the tree on `main` checks it once a second and asks for a new one only when the goal changes, more than 4.0 m of path remain or the rest of the path is blocked (Nav2's default tree);
- **the local costmap:** a grid map that moves with the robot and stores in each cell a cost, highest where an obstacle sits, like an occupancy grid (G-2340) with costs in place of probabilities (layered costmaps, RO-043, a later Note). Nav2's bring-up file makes it 3 m by 3 m with 0.05 m cells, 3600 cells (Nav2 `nav2_params.yaml`);
- **the robot's pose and speed:** at 2.4 s the robot is at (2.387, 1.860), facing 74.33°, at 0.5 m/s;
- **two optional topics:** `obstacles`, obstacles given as shapes by another program, and

`via_points`. A point the band should pass near, pulled on by a cost term of weight `weight_viapoint`, is a **via-point** (G-3295); via-points come from that topic or are taken from the global plan every `global_plan_viapoint_sep` metres, which is off by default (teb_local_planner). Section 10 tries them.

The controller runs every 0.1 s, 10 times a second, as in RO-058 and in TEB's published benchmark (Rösmann et al. 2021); Nav2's controller server defaults to 20 Hz (Nav2 `nav2_params.yaml`) and the package's example file to 5 Hz.

2.3 The parameters, and the six we change

In ROS 2 every parameter the file does not set keeps the value written in the constructor of `TebConfig` in `include/teb_local_planner/teb_config.h` (ros2-master); ROS 1 took its defaults from `cfg/TebLocalPlannerReconfigure.cfg`, and several differ, such as `max_global_plan_lookahead_dist`, 1.0 m in the C++ file and 3.0 m in the ROS 1 file. We use the C++ defaults and change six:

- `max_global_plan_lookahead_dist` 3.0 m (C++ default 1.0 m), as in the ROS 1 file and the package's Nav2 example file;
- `max_vel_x` 0.5 m/s and `max_vel_theta` 1.0 rad/s (defaults 0.4 m/s and 0.3 rad/s), our robot's limits, explained below;
- `acc_lim_x` 0.5 m/s^2, our robot's, the same as the default;
- `acc_lim_theta` 10 rad/s^2 (default 0.5): our robot's model changes its turn rate at once, so we set this limit high enough never to bind;
- `min_obstacle_dist` 0.1 m (default 0.5 m), RO-058's clearance for the small room;
- `xy_goal_tolerance` 0.02 m (default 0.2 m): in Nav2 the goal checker passes its tolerance in, and we use RO-058's arrival test, within 0.02 m and 0.05 rad, stopped.

Two limits for a robot with one. TEB limits the speed and the turn rate separately, while our robot's limit is on each wheel. The wheel speeds for a forward speed and a turn rate are, with:

- v, the forward speed, in m/s;
- ω, the turn rate, in rad/s;
- L, the wheel separation, 0.2 m;
- v_R, v_L, the right and left wheel speeds, in m/s.

$$v_R = v + \omega L/2$$

$$v_L = v - \omega L/2$$

Keeping both at most 0.6 m/s in size gives a diamond of allowed commands in the plane of speed and turn rate, while TEB's two limits give a box (Figure 4). TEB never commands more than the wheels can deliver when the box's corner lies on the diamond. With:

- $v_{\max}$, TEB's speed limit `max_vel_x`, in m/s;

- $\omega_{\max}$, TEB's turn-rate limit `max_vel_theta`, in rad/s;
- v_w, each wheel's limit, 0.6 m/s.

$$\text{faster wheel} = 0.5 + 1.0 \times 0.2/2$$

$$= 0.5 + 0.1$$

$$= 0.6 \text{ m/s}$$

$$v_{\max} + \omega_{\max} L/2 = v_w$$

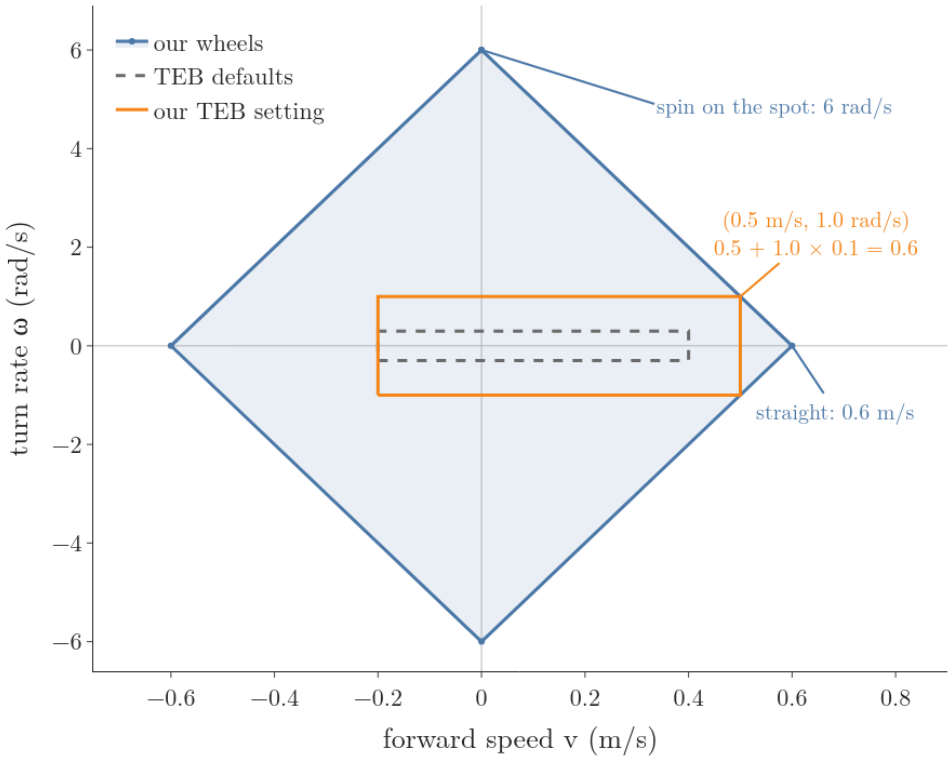


Figure 4: The commands our robot can drive and the ones TEB may command, in the plane of forward speed v (across) and turn rate ω (up). Blue diamond: each wheel at most 0.6 m/s, so 0.6 m/s straight ahead or 6 rad/s on the spot. Grey dashed: TEB's default box, 0.4 m/s and 0.3 rad/s, with reverse down to −0.2 m/s. Orange: our setting, 0.5 m/s and 1.0 rad/s, whose corner touches the diamond: $0.5 + 1.0 \times 0.1 = 0.6$

The cost of this choice is speed: TEB plans at most 0.5 m/s on straight pieces where the wheels allow 0.6 m/s, and at most 1 rad/s on the spot where they allow 6 rad/s. The inputs and settings are fixed; the first thing the code does with them is decide how much of the plan to use.

3. Step 1: the plan ahead and the local goal

> **Key point:** TEB drops the plan behind the robot, keeps it from the pose nearest the robot to the first pose more than 85 percent of half the costmap's width away, 1.275 m in a 3 m costmap, and makes that pose the local goal, facing the average direction of the plan's next three pieces. At 2.4 s that is 32 poses and a local goal 1.29 m ahead, facing 90°.

The opening's plan is 3.698 m long, but the band can only avoid what the costmap shows, at most 1.5 m to each side of the robot. TEB needs a goal inside that window.

The idea. A driver who follows a long route on a navigation display aims at the farthest point the display shows. TEB does the same, with the local costmap as the display. RO-058 called the pose it aims at the local goal (G-2929) and took it 1.5 m along the route; the code takes it differently (teb_local_planner, `pruneGlobalPlan()`, `transformGlobalPlan()`, `estimateLocalGoalOrientation()`):

1. **prune:** drop the plan's poses before the first one within `global_plan_prune_distance`, 1 m, of the robot; at 2.4 s the robot is still within 1 m of the plan's start, so nothing is dropped;
2. **cut:** from the plan's pose nearest the robot, keep poses until one lies further from the robot than 85 percent of half the costmap's width, or until the kept plan is longer than `max_global_plan_lookahead_dist`, 3 m; that pose is kept and is the local goal;
3. **turn:** face the local goal along the average direction of the plan's next three pieces after it; near the plan's end, use the end pose's own heading.

Figure 5 shows the cut at 2.4 s.

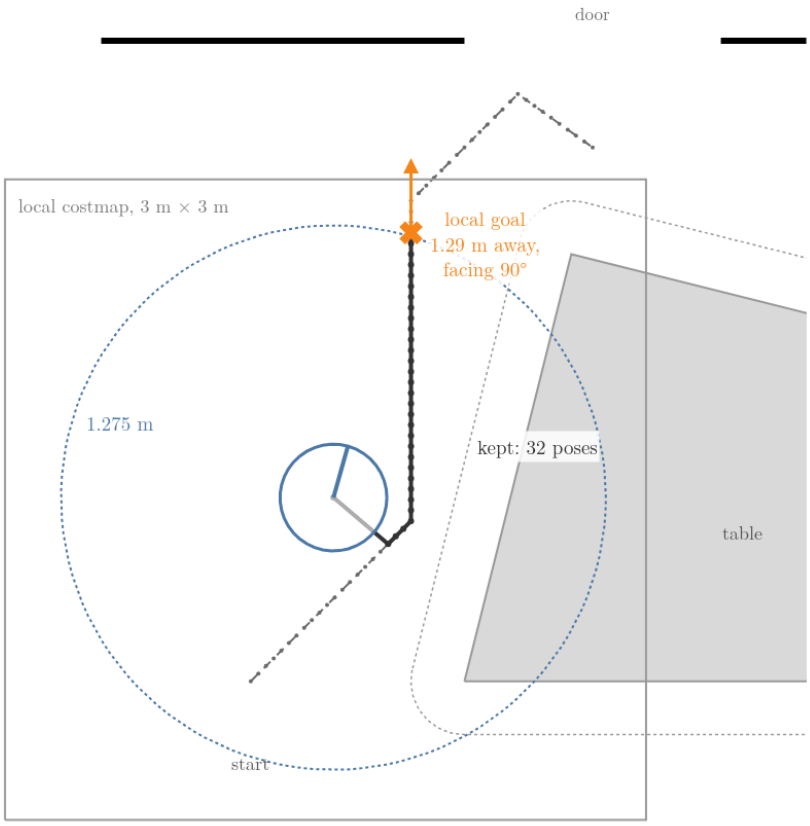


Figure 5: Step 1 at 2.4 s, in our Python version of TEB's algorithm (not the package running). Grey dashed: the global plan, a pose every 0.05 m. Thin grey square: the local costmap, 3 m by 3 m round the robot (blue). Blue dotted circle: 1.275 m from the robot. Dark: the 32 poses TEB keeps, from the pose nearest the robot to the first pose outside the circle. Orange cross and arrow: the local goal, 1.29 m away, facing 90°, the direction of the plan beyond it

The distance at which the cut happens, with:

- s, the costmap's side, 3 m;
- d_{cut}, the largest distance from the robot at which a plan pose is still taken, in m.

$$d_{\text{cut}} = 0.85 \times 3/2$$

$$= 1.275 \text{ m}$$

$$d_{\text{cut}} = 0.85\, s/2$$

The factor 0.85 keeps the local goal off the costmap's border, so that obstacles near the border still lie beyond it (teb_local_planner, comment in `transformGlobalPlan()`). The

pose before the local goal is 1.2439 m away, the local goal 1.2918 m away: the first one past 1.275 m (notebook). Then TEB writes the robot's own pose over the first kept pose, so the kept plan runs from the robot to the local goal.

A pitfall. In a 3 m costmap the local goal is never more than about 1.3 m ahead, whatever `max_global_plan_lookahead_dist` says above that; and the C++ default of 1.0 m would cut the plan sooner than the costmap does. To plan further ahead, the costmap must grow, which costs time (Section 4). The local goal is set; next, the obstacles the band must keep away from.

4. Step 2: obstacles from the costmap

> **Key point:** TEB does not get the room's shapes: every lethal cell of the local costmap becomes a point obstacle, 534 of them at 2.4 s. Each pose of a band then reads only a few of them, the nearest on its left and on its right and any very close, through the robot's footprint model, here a circle of radius 0.25 m.

The opening's 590 cells are the only information TEB has about the person, the table and the walls. This step turns them into obstacles the optimiser can measure distances to.

4.1 Cells become point obstacles

The idea. Consider the costmap as squared paper with the blocked squares shaded. TEB marks the centre of every shaded square and treats each mark as an obstacle of zero size.

The standard terms. A costmap cell that holds an obstacle, with the highest cost, 254 in Nav2, so that a robot centred on it would certainly collide, is a **lethal cell** (G-3290) (navigation2, `cost_values.hpp`). An obstacle given as one point, the centre of a lethal cell, which the optimiser keeps the robot's footprint away from, is a **point obstacle** (G-3291) (teb_local_planner). TEB's default has no other step: `updateObstacleContainerWithCostmap()` adds one point obstacle per lethal cell, except cells behind the robot and further than `costmap_obstacles_behind_robot_dist`, 1.5 m (teb_local_planner).

At 2.4 s (Figure 6):

- 590 lethal cells: the part of the table inside the window, the pole, and the 60 cells of the person, who appeared this cycle;
- 26 of them lie behind the robot and more than 1.5 m away, and are dropped;
- 534 point obstacles are left.

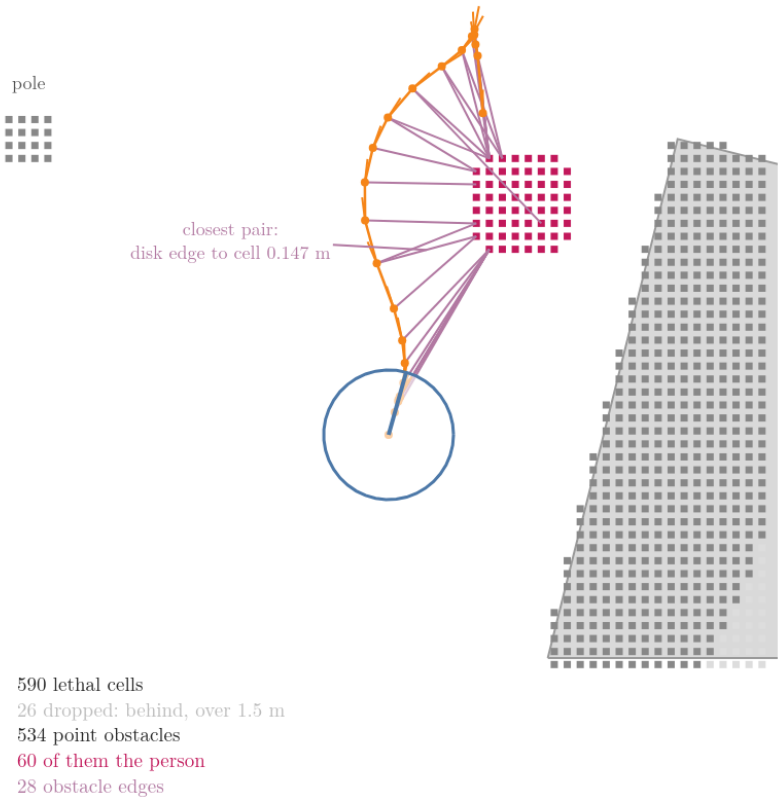


Figure 6: Step 2 at 2.4 s, in our Python version of TEB's algorithm (not the package running). Grey squares: lethal cells of the 3 m costmap (pale: the 26 behind the robot and over 1.5 m away, dropped); pink: the person's 60 cells. Each remaining cell is a point obstacle, 534. Orange: the band TEB picks in this cycle, after optimising. Purple lines: the 28 pairs of a pose and an obstacle point that the band's obstacle terms use. The closest pair leaves 0.147 m between the robot's disk and the cell

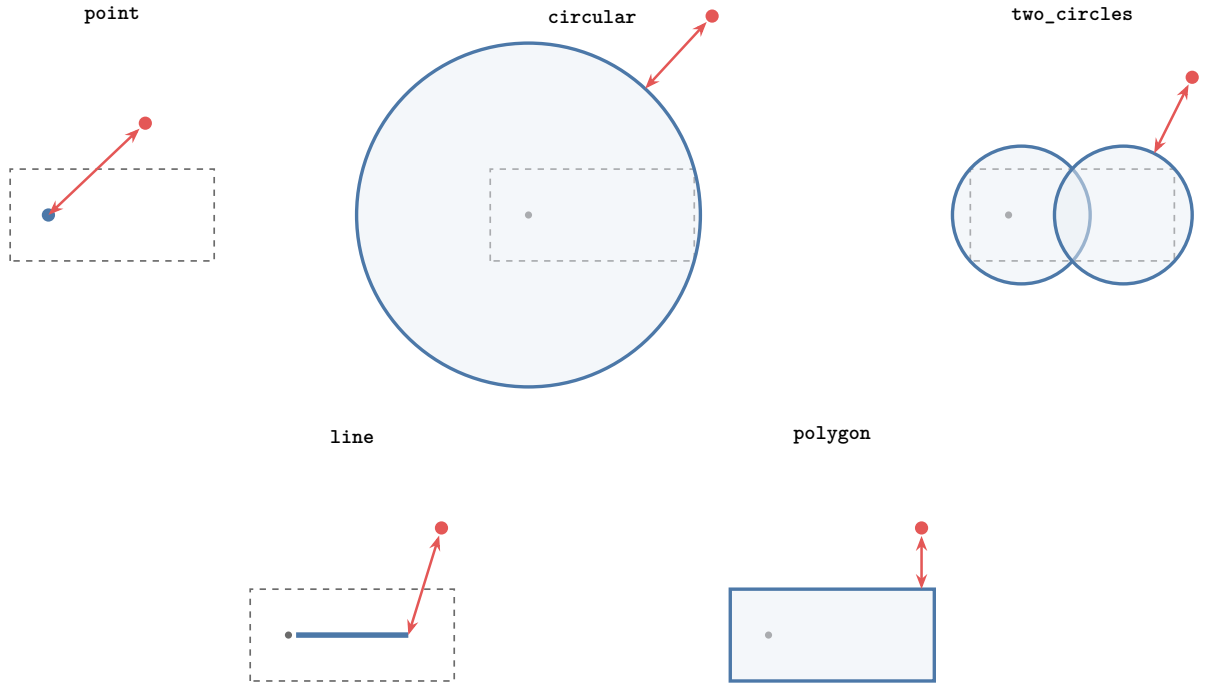
A pitfall. The number of points grows with the costmap's area and falls with coarser cells, and TEB's computing time grows with it; the package's FAQ names the costmap's size and resolution as the main cost (TEB FAQ). A **costmap converter** (G-3292) is a plugin that groups neighbouring lethal cells into a few polygons or lines and hands those to TEB instead of the points; the package's Nav2 example file loads `CostmapToPolygonsDBSMCCH`, which groups cells within 0.4 m of each other and takes the convex polygon round each group (costmap_converter, `costmap_to_polygons.cpp`). We keep the points, the code's default.

4.2 The footprint model and which obstacles each pose reads

The idea. To keep the robot off a point, TEB needs the distance from the robot's body to the point. The body can be described coarsely or exactly; coarser is faster.

The standard term. The shape TEB uses for the robot when it measures the distance to an obstacle, one of a point, a circle, two circles, a line or a polygon, is the **footprint model**

(G-3293) (teb_local_planner, `robot_footprint_model.h`). Figure 7 draws all five round RO-055's car. Our robot is a disk, so we use the circle of radius 0.25 m: the distance to an obstacle point is the distance from the robot's centre minus 0.25 m.



dashed: the car of RO-055 (4.0 m × 1.8 m); grey dot: its pose, the middle of the rear axle; blue: the model TEB measures from (the line has no width, so min_obstacle_dist must cover the car's half width); red: the distance to an obstacle point that the obstacle edges use. In the order point, circular, two_circles, line, polygon the models cost more to compute; the polygon is the exact outline.

Figure 7: TEB's five footprint models (parameter footprint_model.type), drawn round RO-055's car, 4.0 m by 1.8 m (dashed), whose pose is the middle of its rear axle (grey dot). Blue: the model; red: the distance to an obstacle point that the obstacle terms use. point: to the pose; circular: to the pose minus a radius, 3.37 m to cover the car; two_circles: the nearer of two circles along the car; line: to a segment, with no width, so min_obstacle_dist must cover the half width; polygon: to the outline. In that order they cost more to compute, and the polygon is exact

A band of 24 poses and 534 points would make over 12 000 pairs. TEB keeps few: for each pose except the first and the last it takes (teb_local_planner, `AddEdgesObstacles()`):

- every point closer than 1.5 times `min_obstacle_dist`, 0.15 m from the footprint;
- of the rest within 5 times `min_obstacle_dist`, 0.5 m, the nearest one on the pose's left and the nearest on its right.

The band picked at 2.4 s keeps 28 pairs (Figure 6). Each pair adds two penalties, with:

- d, the distance from the robot's disk to the obstacle point, in m, 0.1473 for the closest pair;

- $d_{\min}$, `min_obstacle_dist`, 0.1 m;
- ϵ, `penalty_epsilon`, the margin added to every limit, 0.05 m;
- d_{inf}, `inflation_dist`, 0.6 m;
- e_{obs}, how far the disk comes within the clearance plus the margin, in m;
- e_{inf}, how far it comes within the inflation distance, in m.

$$e_{\text{obs}} = 0.1 + 0.05 - 0.1473$$

$$= 0.0027 \text{ m}$$

$$e_{\text{inf}} = 0.6 - 0.1473$$

$$= 0.4527 \text{ m}$$

$$e_{\text{obs}} = \max(0,\ d_{\min} + \epsilon - d)$$

$$e_{\text{inf}} = \max(0,\ d_{\text{inf}} - d)$$

Each is squared and weighted: the first by `weight_obstacle`, 50, giving 0.0004; the second by `weight_inflation`, 0.1, giving 0.0205 (notebook). The second term is small but reaches out to 0.6 m, so it moves the band slightly away from obstacles even when the clearance holds (teb_local_planner, `EdgeInflatedObstacle`). The obstacles are ready; next, the bands that will be pushed away from them.

5. Step 3: warm start, or start again

> **Key point:** Each band from the last cycle is moved to the robot and the new local goal and kept as the first guess, unless the local goal has jumped 1 m or turned 90° or more, in which case every band is dropped. A band is also built from the plan, a pose at every plan point, timed at full speed. At 2.4 s both first guesses run through the person.

The opening's band must be found in one cycle; an optimiser needs a first guess, and TEB has two kinds.

The warm start. The first kind is the band of the last cycle, a warm start (G-2914): RO-058's loop drops the poses the robot has passed, moves the first pose to the robot and the last to the new local goal (teb_local_planner, `updateAndPruneTEB()`). The package adds a rule RO-058 did not need (`updateAllTEBs()`): if the new local goal lies 1.0 m or more from the old one (`force_reinit_new_goal_dist`), or faces 90° or more away from it

(`force_reinit_new_goal_angular`, $\pi/2$ on ros2-master, 0.78 rad in ROS 1), every band is dropped, because a band shaped for one goal is a poor first guess for a very different one.

In our drive the rule fires once (Figure 8, bottom). Until 4.6 s the local goal faces along the plan's last piece, which heads down to the door pose at $-35.54°$; at 4.7 s the local goal, now 0.143 m from the plan's end, takes the door pose's own heading, 90° (Section 3, step 3) (notebook):

$$90 - (-35.54) = 125.54°$$

which is more than 90°, so the bands are dropped and a new one is built from the plan. In the clear room the same happens at 4.1 s.

The band from the plan. The second kind is built each cycle from the kept plan (`initTrajectoryToGoal()`): a pose at every plan point, facing the next point, and between neighbours the time to drive there at `max_vel_x` or to turn there at `max_vel_theta`, whichever is longer. At 2.4 s the first piece runs 0.33572 m from the robot to the plan while turning 0.51196 rad, with:

- ℓ, the piece's length, in m;
- $\Delta\theta$, its turn, in rad;
- ΔT, its time step, in s.

$$\Delta T = \max(0.33572/0.5,\ 0.51196/1.0)$$

$$= \max(0.67144,\ 0.51196)$$

$$= 0.67144 \text{ s}$$

$$\Delta T = \max(\ell/v_{\max},\ |\Delta\theta|/\omega_{\max})$$

The plan's other pieces, 0.05 m long, take 0.1 s each. Before optimising, TEB resizes the band so that neighbours are about `dt_ref`, 0.3 s, apart, within `dt_hysteresis`, 0.1 s. On ros2-master the rule is the older one (`autoResize()`): halve any step longer than 0.4 s by putting a pose in its middle, and merge any step shorter than 0.2 s into the next; ROS 1's `noetic-devel` trims steps between 0.4 and 0.6 s instead of halving them, the rule RO-058 described. The 32 poses become 17, with steps from 0.2 to 0.4 s (notebook).

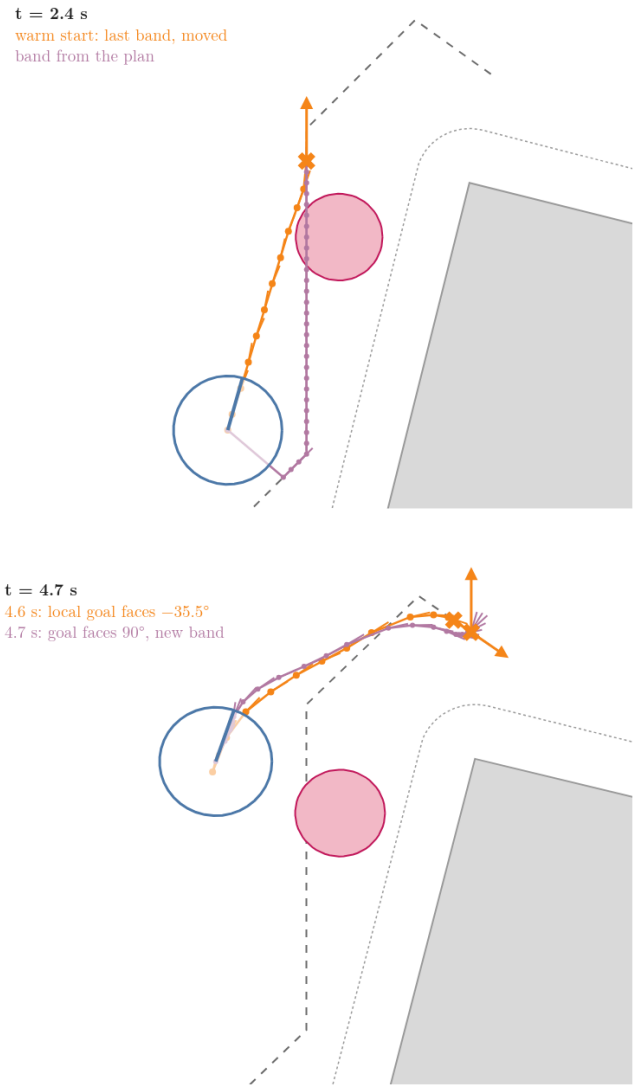


Figure 8: Step 3, in our Python version of TEB's algorithm (not the package running). Top, 2.4 s: the two first guesses, before any optimising. Orange: last cycle's band (12 poses), its first pose moved to the robot and its last to the local goal; purple: the band built from the plan, a pose at every plan point. Both run through the person, who was not there when they were made. Bottom: at 4.6 s the local goal (orange cross, left) faces −35.5° along the plan's last piece (orange: that cycle's band); at 4.7 s the local goal, near the plan's end, takes the door pose's heading, 90°, a turn of 125.5°, so the bands are dropped and a new one is built from the plan (purple, after optimising)

Both first guesses of 2.4 s run through the person. An optimiser started inside an obstacle is pushed out by the shortest way, and RO-058 showed that the side it is pushed to depends on where it starts; the next step decides how many sides to try.

6. Step 4: candidates in different homotopy classes

> **Key point:** One band cannot cross an obstacle to its other side, so TEB keeps up to 5 bands, one per homotopy class. TEB tells classes apart by the H-signature, one number per obstacle whose sign says on which side the band passes, and finds new classes by searching a roadmap of 15 random points depth-first. With the person 0.25 m further left, this gives bands on both sides.

The opening asks which side of the person to take. RO-058 left this to the warm start, and with the person 0.25 m further left, at (2.65, 2.75), the warm start entered the narrow gap on the right and overlapped the person.

The standard term. Two paths belong to the same homotopy class (G-2922) when one can be bent into the other without crossing an obstacle. Keeping one band in each of several homotopy classes, optimising them all and picking the cheapest is **homotopy-class planning** (G-3294), which TEB's authors call planning in distinctive topologies (Rösmann et al. 2015).

6.1 Telling the sides apart: the H-signature

The idea. An observer at the person's position who follows the robot with their eyes turns through about half a turn in one direction when the robot passes on the left, and in the other direction when it passes on the right. The direction of that turn distinguishes the two paths.

The mechanism on numbers. Figure 9 does this for two bands at 2.5 s, with the person 0.25 m further left. Watch the two dotted lines from the person's centre: the orange one, to the band on the left, turns clockwise; the purple one, to the band on the right, anticlockwise.

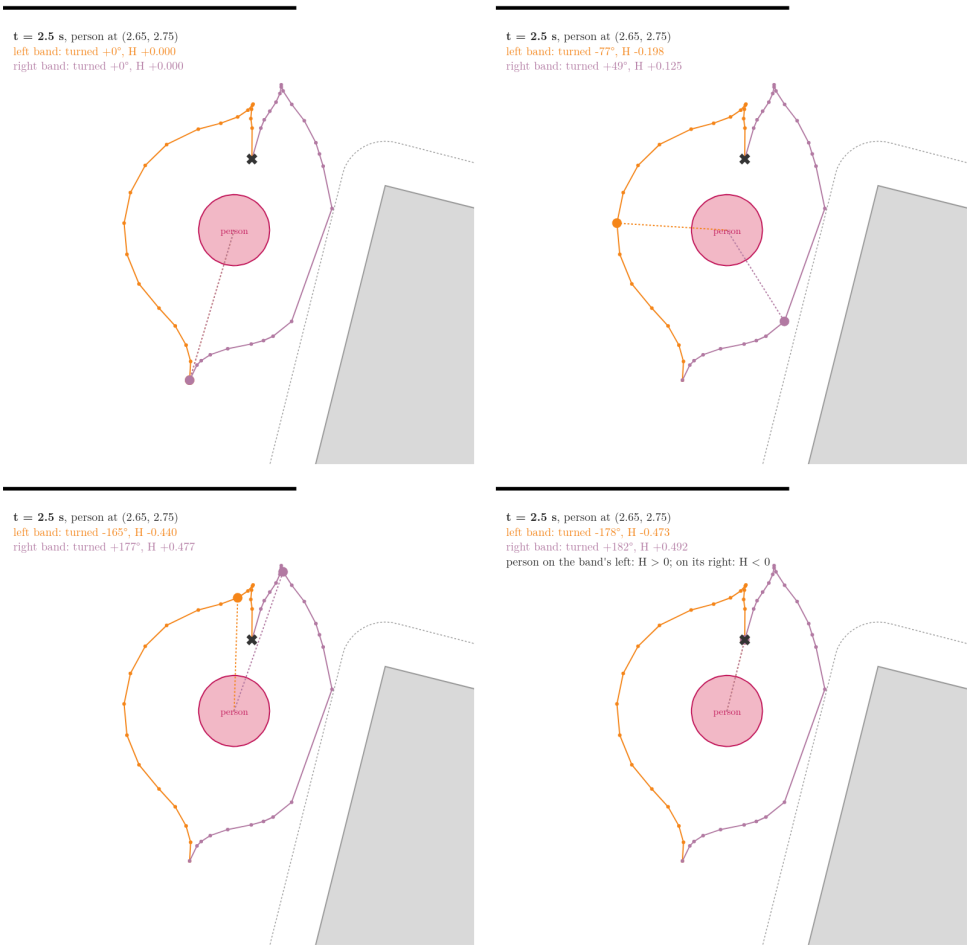


Figure 9: How TEB tells two bands apart, with the person 0.25 m further left (pink), in the cycle at 2.5 s, in our Python version of TEB's algorithm (not the package running). Orange: a band passing left of the person; purple: one passing right. A dot runs along each band and a dotted line joins it to the person's centre. The text adds up the angle each line has turned through (anticlockwise positive) and the band's H-signature so far for the person's centre, the number the code computes. Last frame: the left band turned $-177.5°$, $H = -0.473$; the right band $+182.5°$, $H = +0.492$. Both bands bend back to the local goal (cross), which sits just above the person

The left band turns the line through $-177.5°$ and the right band through $+182.5°$ (notebook): opposite signs, two classes.

The standard term. The number TEB computes for each obstacle from the angle a band turns through as seen from it, positive when the obstacle lies on the band's left, is the **H-signature** (G-3296) (teb_local_planner, `h_signature.h`; Rösmann et al. 2015). The code (`HSignature3d`, used by default because `include_dynamic_obstacles` is true) treats the band as a line in space and time and each obstacle as an upright line, and adds up a field round that line along the band; for an obstacle that does not move, each small step of the band adds the angle it turns through, weighted by a factor between 1 and 2 that grows with

the time already spent, and the total is divided by 4π.

A check at easy values. A straight line 6 m long passes an obstacle at 0.4 m, from (−3, 0.4) to (3, 0.4), with the obstacle at the origin. With:

- the start angle and the end angle, the directions from the obstacle to the line's start and end, in rad, anticlockwise from the x axis;
- ϕ, the angle the line from the obstacle turns through, the end angle minus the start angle, in rad;
- H, the H-signature, with no time spent so the weight is 1.

$$\text{start angle} = \pi - \arctan(0.4/3)$$

$$= 3.00904 \text{ rad}$$

$$\text{end angle} = \arctan(0.4/3)$$

$$= 0.13255 \text{ rad}$$

$$\phi = 0.13255 - 3.00904$$

$$= -2.87649 \text{ rad}$$

$$H = -2.87649/(4\pi)$$

$$= -0.2289$$

$$H = \phi/(4\pi)$$

The line turns through −164.81°, clockwise, because the obstacle lies on its right.

The code gives −0.2289 for this line sampled at 61 points (notebook); passing below the obstacle gives +0.2289. Spending 12 s on the same line raises the weight and gives −0.4519, with the sign unchanged.

The class test. Two bands are in the same class when, for every obstacle that both of them pass closely enough to give a value of at least `h_signature_threshold`, 0.1, the two values have the same sign (`HSignature3d::isEqual()`). With point obstacles, every lethal cell is

an obstacle with its own number. For the person's cells at 2.5 s, the left band gives values from −0.597 to −0.369, the right bands from 0.368 to 0.602 (notebook).

A pitfall of point obstacles. Two bands on the same side of a person can still count as two classes when one cell lies between them. At 2.4 s, in the main scene, both first guesses run through the person (Figure 8, top): the warm band just left of the person's leftmost column of cells, at x = 2.725 m, the plan's band just right of it, on the line x = 2.75 m. For those 6 cells the two bands give opposite signs, so TEB keeps both and optimises both; both end up passing the person on the left (notebook). A costmap converter, which would make the person one polygon, avoids this.

6.2 Finding new classes: a random roadmap

The idea. To find routes past obstacles that no band covers yet, TEB places a few random points between the robot and the local goal, connects those that lie ahead of one another without crossing an obstacle, and lists every path through them.

The mechanism. The steps, in the order the code runs them (`renewAndAnalyzeOldTebs()`, `exploreEquivalenceClassesAndInitTebs()`, `ProbRoadmapGraph::createGraph()`, `DepthFirst()`):

1. compute the class of every band kept from the last cycle, last cycle's best first, and drop a band whose class is already taken;
2. drop a band that sets off more than 90° away from the best band's direction, or takes more than 3 times as long;
3. add the band from the plan if its class is new;
4. scatter `roadmap_graph_no_samples`, 15, random points in a rectangle from the robot to the local goal, `roadmap_graph_area_width`, 6 m, wide;
5. join two points by an edge when it heads forward, its direction within about 63° of the robot-to-goal direction (a cosine above `obstacle_heading_threshold`, 0.45), and passes no point obstacle closer than `min_obstacle_dist`, 0.1 m (the robot's radius is not added here);
6. list every path from the robot to the local goal with depth-first search (G-2552); build a band on each path and keep it if its class is new, until there are `max_number_classes`, 5, bands.

The random points and edges form a small probabilistic roadmap (G-2780), rebuilt every cycle. Figure 10 steps through it at 2.5 s with the person 0.25 m further left.

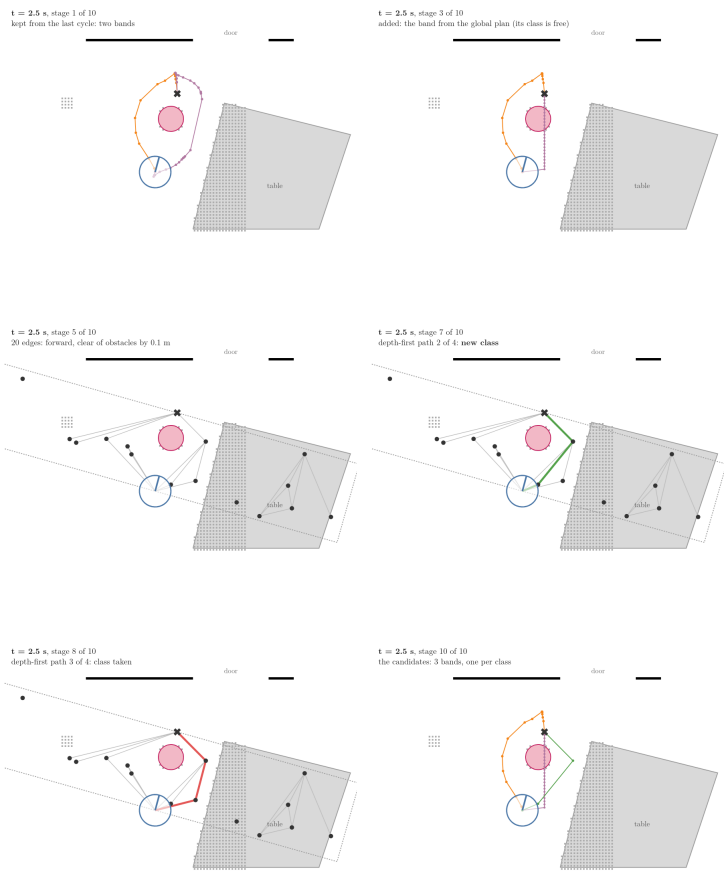


Figure 10: Step 4 stage by stage, person 0.25 m further left, t = 2.5 s, in our Python version of TEB's algorithm (not the package running). Grey squares: lethal cells of the 3 m costmap. 1: two bands kept from 2.4 s, orange on the left (picked then) and purple on the right. 2: the right band sets off 97° away from the left one's direction, more than 90°, and is dropped. 3: the band from the global plan, whose class is now free, is added. 4: 15 random points in a rectangle 6 m wide (dotted). 5: the 20 edges that head forward and pass no point obstacle closer than 0.1 m. 6 to 9: the 4 paths depth-first search finds; red: its class is taken, green: a new class. 10: the three candidates

At 2.5 s (notebook):

- the right band kept from 2.4 s is dropped: it sets off at 22.1°, the left band at 119.4°, 97.3° apart;
- the band from the plan is added: it runs through the right half of the person, a class no band has now;
- of 15 points, 20 edges and 4 paths, the first path is in the left band's class, the second passes wholly right of the person, a new class, and the last two are in the class the second has just taken.

Three candidates go on: one left of the person, two right. In the main scene, at 2.4 s, the

roadmap's 15 points give 20 edges but no path from the robot to the local goal, so the only candidates are the two first guesses (notebook). The candidates are first guesses; next, each is optimised.

7. Step 5: optimise every band: the g2o graph

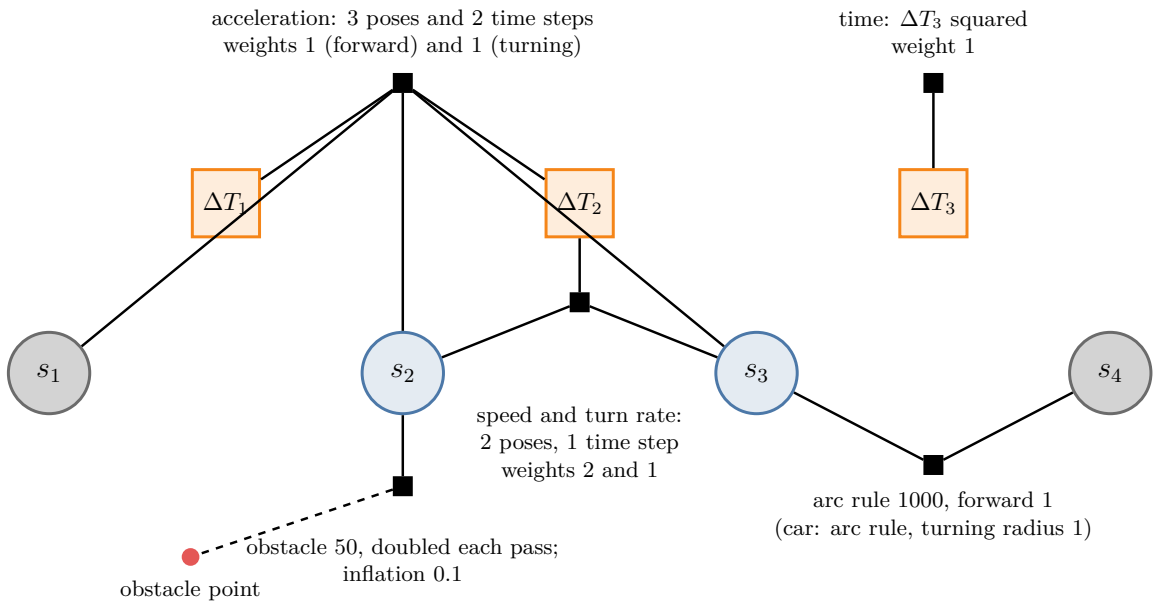
> **Key point:** Each candidate becomes a graph whose vertices are its poses and time steps and whose edges are its cost terms, each joining the few vertices it reads. TEB optimises every band separately, in parallel, with 4 passes of 5 Levenberg–Marquardt steps, and plans each band to stop at its local goal.

The opening's one speed and turn rate will come from a band, but the candidates of Section 6 are unrefined first guesses that still run through or close past the person. This step makes each one fast, smooth and clear.

The cost is RO-058's: the sum of the squared time steps plus squared penalties for every broken limit, time plus penalties, solved by sparse least squares with the Levenberg–Marquardt method (G-2921) in g2o (G-2927). What is new here is how the code builds it.

The idea. The problem resembles beads joined by elastic cords: each pose and each time step is a bead, and each cost term is a cord attached to the beads it depends on. The speed term is attached to two poses and the time between them, the acceleration term to three poses and two time steps. g2o stores the problem in this form.

The standard term. In g2o's graph, one cost term, joined to the vertices it reads, the poses and time steps whose values change its error, is an **edge** (G-3298) of the graph; an edge may join more than two vertices, so the graph is a hypergraph (Rösmann et al. 2013; teb_local_planner, `g2o_types/`). Figure 11 draws one edge of each kind for a band of 4 poses.



grey: fixed (the robot, the local goal); blue and orange: changed by the solver; black square: one edge (one cost term)

Figure 11: The g2o graph TEB builds for a band of 4 poses (TebOptimalPlanner::buildGraph()). Circles: poses s1 to s4, the first and last fixed (grey); squares: time steps ΔT1 to ΔT3; black squares: edges, one cost term each, joined to the vertices it reads. One edge of each kind is drawn with its default weight: time (one time step), speed and turn rate (two poses, one time step), acceleration (three poses, two time steps), arc rule and forward drive (two poses; for a car, the turning radius instead), obstacle (one pose and one obstacle point)

`buildGraph()` adds the edges in this order, each for every pose, piece or pair of pieces it applies to (teb_local_planner, `src/optimal_planner.cpp`):

1. obstacles: one edge per (pose, obstacle point) pair of Section 4.2, weight 50 and 0.1;
2. via-points, when there are any (Section 10);
3. speed and turn rate: one per piece, the penalties for leaving −0.2 to 0.5 m/s and −1.0 to 1.0 rad/s, each limit moved 0.05 inwards, weights 2 and 1;
4. acceleration: one from the robot's measured speed to the first piece, one per pair of pieces, one from the last piece to a stop at the local goal, weights 1 and 1;
5. time: one per time step, weight 1;
6. shortest path: weight 0 by default, so none;
7. the arc rule and forward driving, one per piece, weights 1000 and 1, or for a car the arc rule and the turning radius (Section 11);
8. a preferred turning direction, only during oscillation recovery (Section 9).

The band picked at 2.4 s has 24 poses and 23 time steps, 89 numbers to choose, and 121 edges: 23 time, 23 speed and turn rate, 24 acceleration, 23 arc rule, 28 obstacle (notebook). Item 4 contains a pitfall: `free_goal_vel` is false by default, so every cycle each band plans

to stop at its local goal, about 1.3 m ahead; the stop keeps moving away as the robot drives.

Solving. Each band is optimised in its own thread, with no link between bands (`optimizeAllTEBs()`). Each runs `optimizeTEB()`: 4 passes, each of which resizes the band, builds the graph (the obstacle pairs are chosen here), takes 5 Levenberg–Marquardt steps and doubles the obstacle weight. g2o sets the step size by its own rule (g2o, `optimization_algorithm_levenberg.cpp`), which our version copies: at the start of each pass the damping, the amount added to the diagonal of the matrix each step solves, is 0.00001 times its largest diagonal entry, the same for every number; after a step that lowers the cost it shrinks by a factor between 1/3 and 2/3, and after one that does not it grows, by 2, then 4, then 8, for up to 10 tries.

Figure 12 runs the 4 passes for the three candidates at 2.5 s, with the person 0.25 m further left. Watch the two right-hand bands move into the gap between the person and the table while the left band becomes smooth round the person.

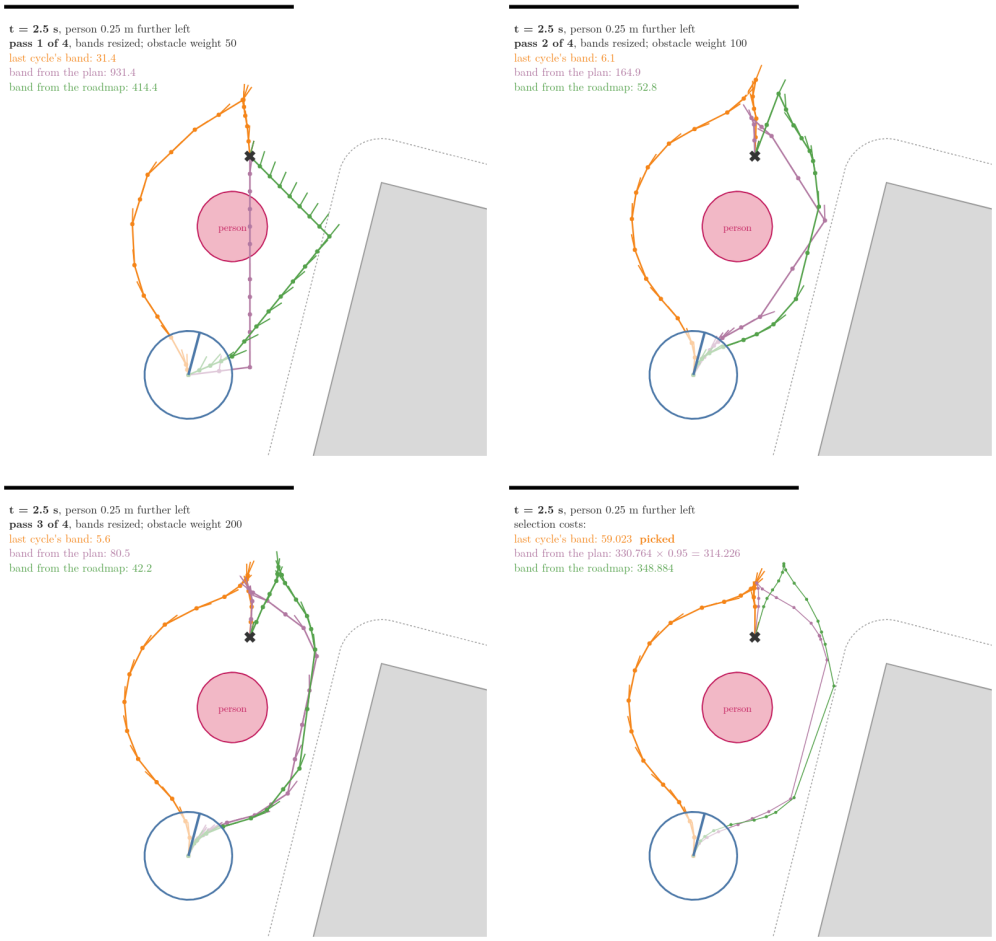


Figure 12: Steps 5 and 6, person 0.25 m further left, t = 2.5 s, in our Python version of TEB's algorithm (not the package running): the three candidates optimised side by side, one Levenberg–Marquardt step per frame. Orange: last cycle's band, left of the person; purple: the band from the plan, right; green: the band from the roadmap, right. Each pass resizes the bands, then takes 5 steps; the obstacle weight starts at 50 and doubles each pass. The text gives each band's cost with the default weights. Last frame: the selection costs; the band from the plan's cost is multiplied by 0.95; the left band, 59.023, is picked

With the default weights, the left band's cost falls from 31.37 to 2.42, the plan's band from 931.43 to 8.62 and the roadmap's band from 414.37 to 18.41 (notebook). Three optimised bands, one to send.

8. Step 6: pick the cheapest band

> **Key point:** TEB scores each optimised band by the sum of all its edges' weighted squared errors, with the obstacle edges multiplied by 100, and multiplies the score of the band in the global plan's class by 0.95. The band with the lowest score is chosen: with the person 0.25 m further left, the band on the left, 59.023 against 314.226 and 348.884.

The opening's question of which side ends here: the cycle must send the speed of one band.

The idea. Every band receives one score: time adds a small amount, every exceeded limit more, and proximity to an obstacle the most. The band in the global plan's class receives a 5 percent reduction, so that between bands of nearly equal score TEB keeps to the global plan's side (the code's comment calls it a small hysteresis), and the band with the lowest score is chosen.

The mechanism. After the last pass, `computeCurrentCost()` adds up the weighted squared errors of every edge of the band, with the obstacle weight of that pass, 50 doubled three times, 400, and with every obstacle edge multiplied by `selection_obst_cost_scale`, 100. `selectBestTeb()` then multiplies (teb_local_planner, `src/homotopy_class_planner.cpp`):

- the cost of last cycle's best band by `selection_cost_hysteresis`, 1.0, so no hysteresis by default;
- the cost of the band in the global plan's class by `selection_prefer_initial_plan`, 0.95;

and picks the lowest. At 2.5 s with the person 0.25 m further left (notebook):

$$\text{plan's band} = 330.764 \times 0.95$$

$$= 314.226$$

$$59.023 < 314.226 < 348.884$$

The left band has by far the lowest score: the two right-hand bands pass too close to the person or the table, and the obstacle terms, multiplied by 400 and then 100, dominate. The choice fixes the opening's left-person case: the robot leaves on the left in the cycle after the person appears, where RO-058's single band stayed on the right.

In the main scene at 2.4 s the two candidates both pass left; the plan's band is chosen because of its 5 percent reduction:

$$60.76 \times 0.95 = 57.72$$

$$57.72 < 62.23$$

Before the picked band is used, it is checked.

9. Step 7: check, command and failure

> **Key point:** TEB checks the robot's footprint at the band's first 6 poses against the costmap's lethal cells; if any touches, it drops all bands and sends nothing. Otherwise it sends the speed and turn rate of the band's first piece, each limited to its bound. Repeated failures shorten the horizon; oscillation makes it prefer one turning direction.

The opening's cycle ends with one command; this step produces it, or rejects the band.

The feasibility check. The band's obstacle terms are soft: a band can still touch an obstacle when breaking the clearance costs less than the alternative (RO-058, hard and soft constraints). So before sending anything, TEB tests the robot's real footprint against the costmap at the robot's pose and the next `feasibility_check_no_poses`, 5, poses, with extra poses in between wherever neighbours are further apart than the robot's inscribed radius, 0.25 m here (`isTrajectoryFeasible()`). Testing a band's first poses against the costmap's lethal cells before it is used, and rejecting it if the footprint touches one, is the **feasibility check** (G-3297) (teb_local_planner). Only the first poses are tested because the robot drives only the first piece before the next cycle plans again.

At 2.4 s the six disks are at least 0.414 m from every lethal cell, so the band passes (Figure 13).

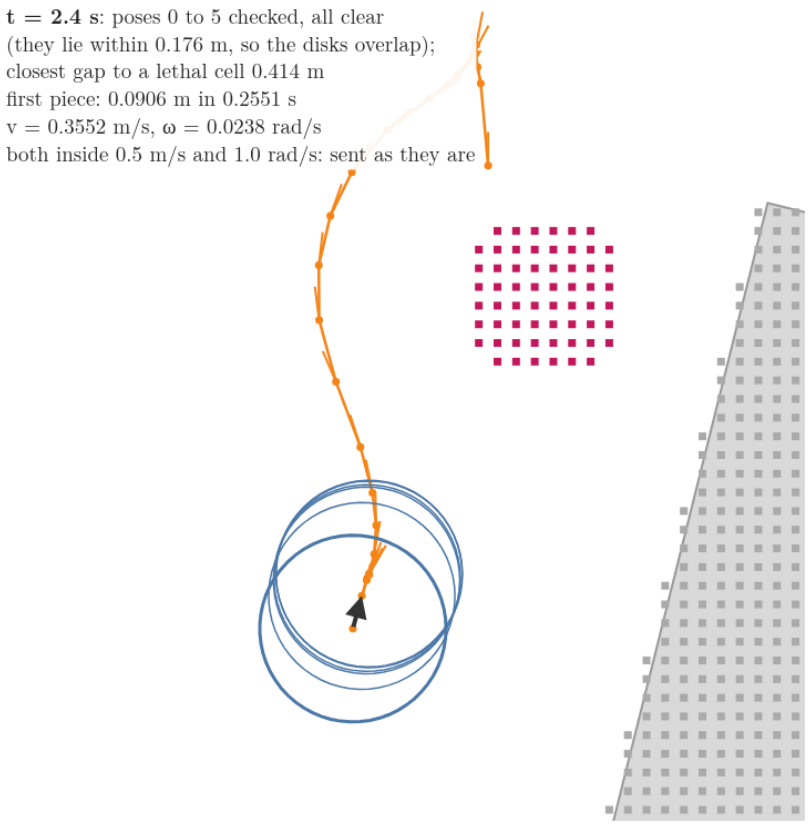


Figure 13: Step 7 at 2.4 s, in our Python version of TEB's algorithm (not the package running). Orange: the band TEB picked. Blue circles: the robot's disk at poses 0 to 5, which the feasibility check tests against the lethal cells (grey; the person pink); all are at least 0.414 m clear. The first poses lie close together, within 0.176 m, so the disks overlap. Dark arrow: the band's first piece, 0.0906 m in 0.2551 s, which gives the command, 0.3552 m/s and 0.0238 rad/s

The command. TEB sends the speed and turn rate of the band's first piece (`getVelocityCommand()` with `control_look_ahead_poses` 1), with:

- ℓ_1, the first piece's length, 0.09061 m;
- $\Delta\theta_1$, its turn, 0.00607 rad;
- ΔT_1, its time step, 0.25511 s.

$$v = 0.09061/0.25511$$

$$= 0.3552 \text{ m/s}$$

$$\omega = 0.00607/0.25511$$

$$= 0.0238 \text{ rad/s}$$

$$v = \ell_1 / \Delta T_1$$

$$\omega = \Delta\theta_1 / \Delta T_1$$

The band slows the robot from 0.5 m/s while it swerves. Then `saturateVelocity()` limits each: the speed to at most 0.5 m/s forward and 0.2 m/s backward, the turn rate to 1.0 rad/s, each on its own. The limiting is needed because the band's limits are soft: at 0.7 s the band's first piece is 0.12509 m in 0.24825 s, 0.5039 m/s, and the robot is sent 0.5 m/s; in 62 of the drive's 98 cycles the band's first piece exceeds 0.5 m/s (notebook). The command goes to Nav2's velocity smoother and collision monitor, then to the wheels.

When there is no command. In five cases TEB drops all its bands, so the next cycle starts again, and throws an error to the controller server instead of a command (`computeVelocityCommands()`):

- the planner fails;
- the solution diverged, a test that is off by default (`divergence_detection_enable`);
- the feasibility check fails;
- the first piece gives no valid command, such as a time step of 0;
- for a car, the steering angle is not finite.

The controller server stops the robot; if the failures last longer than its `failure_tolerance`, 0.3 s in Nav2's bring-up file, it aborts the path-following action, and Nav2's behaviour tree decides what to do, such as clear the costmap or back up (Nav2 `nav2_params.yaml`; behaviour trees and recovery). Before planning, TEB also runs two backups of its own (`configureBackupModes()`):

- **shorter horizon:** after an infeasible cycle, and for at least `shrink_horizon_min_duration`, 10 s, it keeps only the nearer half of the plan ahead, so the band has less to solve;
- **oscillation recovery:** it keeps the last 10 s of commands, each divided by its limit; if their mean speed and mean turn rate are both below 0.1 and the turn rate has changed sign more than once, it adds an edge of weight 50 that favours the current turning direction for the first 3 turns, for 10 s (`recovery_behaviors.cpp`).

In our three drives the feasibility check never fails and neither backup starts (notebook). One cycle is complete; the next section runs them all.

10. The whole drive

> **Key point:** Our version of TEB stops on the door pose at 9.8 s with the person, 1.9 s later than in a clear room, against RO-058's 8.0 s and 1.3 s; it plans at 0.5 m/s where RO-058's band reached 0.595 m/s, and spends 1.9 s at the end turning the last 35.5°.

> Our version satisfies Nav2's default goal checker at 8.3 s, against 13.3 s for DWB.

The opening asked for one command in one cycle; 98 cycles later the robot is at the door.

Figure 14 compares the speed of our TEB version with RO-058's band.

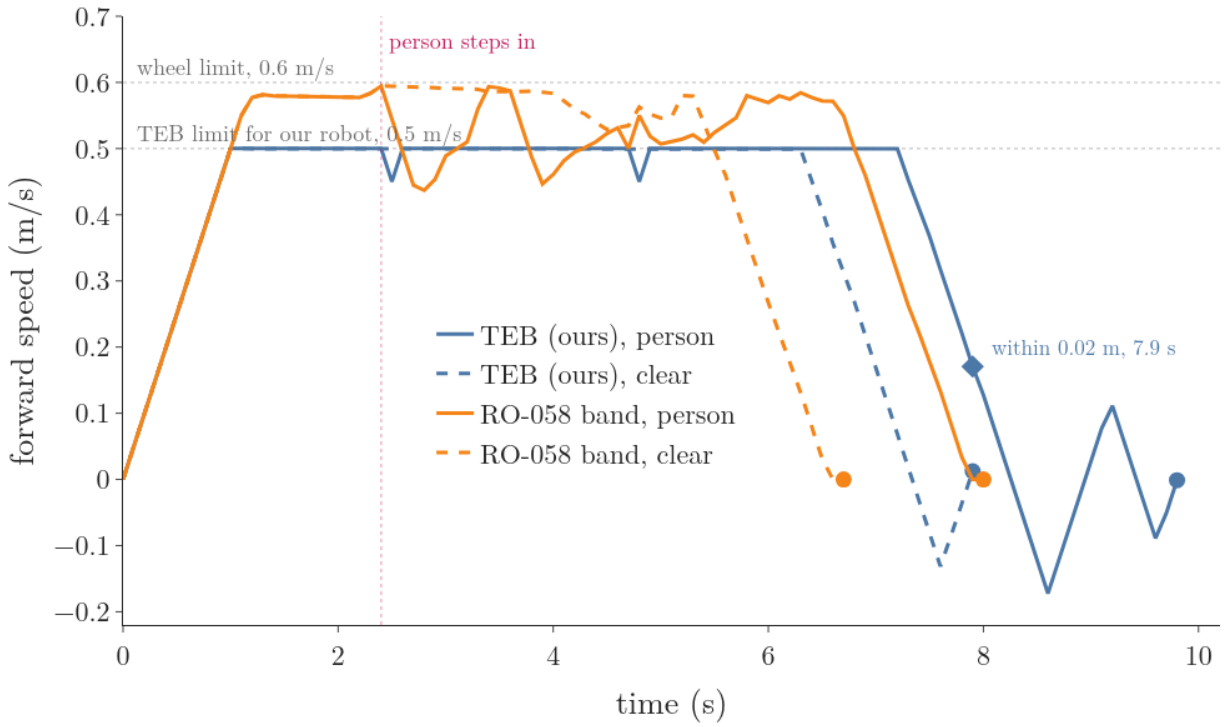


Figure 14: Forward speed over the drive, one value per 0.1 s cycle. Blue: our Python version of TEB's algorithm; orange: RO-058's band. Solid: the person steps in at 2.4 s; dashed: the room clear. Grey dotted: 0.5 m/s, TEB's speed limit for our robot, and 0.6 m/s, the wheel limit RO-058's band used. Dots: arrival on the door pose (within 0.02 m and 0.05 rad, stopped); diamond: our robot first within 0.02 m of the door pose, 7.9 s. After that it drives back and forth slowly while it turns

Three things set the times (notebook):

- **the speed limit:** our TEB plans at most 0.5 m/s, RO-058's band up to 0.595 m/s (Section 2.3);
- **the person:** with the person, the robot slows to 0.45 m/s while it swerves and drives 3.709 m instead of 3.197 m: it is within 0.02 m of the door at 7.9 s, against 7.0 s in the clear room;
- **the last turn:** at 7.9 s the robot is on the door pose but faces 35.5° off. To turn the last degrees within 0.05 rad, the bands of 3 poses command small moves forward and back, and the robot stops turned at 9.8 s. Users of the package report the same back-and-forth near the goal when they set tight tolerances (teb_local_planner, issues 195 and 406).

Run	Door pose (RO-058's test)	Nav2's default goal checker	Closest to the person
RO-058 band, clear	6.7 s		
RO-058 band, person	8.0 s		0.14 m
TEB (ours), clear	7.9 s	7.4 s	
TEB (ours), person	9.8 s	8.3 s	0.137 m
TEB (ours), person 0.25 m further left	9.8 s	9.2 s	0.150 m
DWB (RO-078), person, plan made once	aborted at 12.45 s		0.07 m, stopped
DWB (RO-078), person, plan remade each second		13.3 s	0.002 m

The table's goal-checker column is the first moment the robot is within 0.25 m of the door pose and, from then on, within 0.25 rad of its heading, Nav2's default test (Nav2 `nav2_params.yaml`); our TEB version satisfies that test at 8.3 s and DWB at 13.3 s (DWB against TEB). TEB passes the person with the plan made once; DWB needs the plan remade.

The person 0.25 m further left. This was RO-058's failure. Our version of TEB leaves on the left in the cycle after the person appears (Section 8) and passes 0.150 m clear, where RO-058's single band, warm-started on the right, overlapped the person by 0.02 m (notebook).

Via-points. Via-points, points of the plan the band is pulled towards (Section 2.2), are off by default (`global_plan_viapoint_sep` -1); the package's Nav2 example file takes one every 0.3 m of plan with weight 50. Following the plan and being fast are two conflicting aims (TEB tutorial, via-points), and in the room the plan runs through the person (notebook):

Via-points	Within 0.02 m of the door	Mean distance from the plan
none	7.9 s	0.176 m
every 0.3 m, weight 1	9.0 s	0.153 m
every 0.3 m, weight 50	10.4 s	0.118 m

The robot keeps closer to the plan as the weight grows, and arrives later. The differential-drive case is complete; a car cannot turn on the spot, which changes the set-up.

11. Car-like robots

> **Key point:** Setting `min_turning_radius` above 0 swaps TEB's forward-drive edge for a turning-radius edge and lets the band reverse; `cmd_angle_instead_rotvel` turns the command into a steering angle. On RO-055's car turning round, our version finds bands as short as the Reeds–Shepp path, 15.69 m against 15.71 m, and from a planner's path

> with an acceleration limit, a three-point turn in 20.5 s.

The room's robot can spin on the spot, so its band never needs a reversal. A car cannot, so the example changes from the room to RO-055's car on an open floor: 4.0 m by 1.8 m, its axles 2.5 m apart, unable to drive a circle tighter than 5 m (Hybrid A*).

The set-up. Four parameters make TEB plan for a car (teb_local_planner, car-like tutorial; `src/optimal_planner.cpp`):

- `min_turning_radius` 5 m: with any value above 0, `buildGraph()` adds `EdgeKinematicsCarlike` in place of the differential drive's edge: the arc rule as before, weight 1000, and a penalty, weight 1, on each piece whose turning radius (G-2293), its length divided by its turn, is below 5 m; the forward-drive penalty goes, so the band may reverse;
- `max_vel_x_backwards` above 0, so reversing is within the speed limits;
- `wheelbase` 2.5 m and `cmd_angle_instead_rotvel` true, so the command's turn rate becomes a steering angle;
- a footprint model that fits the car (Figure 7), such as two circles or a polygon.

The tutorial advises a turning radius slightly larger than the car's, because the limit is a soft penalty (TEB tutorial, car-like). The steering angle comes from the radius of the first piece, never below 95 percent of `min_turning_radius` (`convertTransRotVelToSteeringAngle()`), with:

- r, the first piece's turning radius, speed over turn rate, in m;
- L_w, the wheelbase, 2.5 m;
- φ, the front wheels' steering angle.

For the first piece of the three-point turn below, 0.18613 m/s and 0.03724 rad/s:

$$r = 0.18613/0.03724$$

$$= 4.998 \text{ m}$$

$$\varphi = \arctan(2.5/4.998)$$

$$= 26.58°$$

$$\varphi = \arctan(L_w/r)$$

the car's largest steering angle, as in RO-055.

Turning round. TEB's car-like paper tests the planner on an open floor: from (2, 0) facing 0° to (−2, 0) facing 180°, a band started as 5 poses on the straight line, no acceleration limits,

speed between −1 and 1 m/s, and compares the band's length with the shortest drivable path, the Reeds–Shepp path (G-2829) (Rösmann et al. 2017, Table I). Figure 15 runs the same test with our car's 5 m radius (test A), then a second test (B).

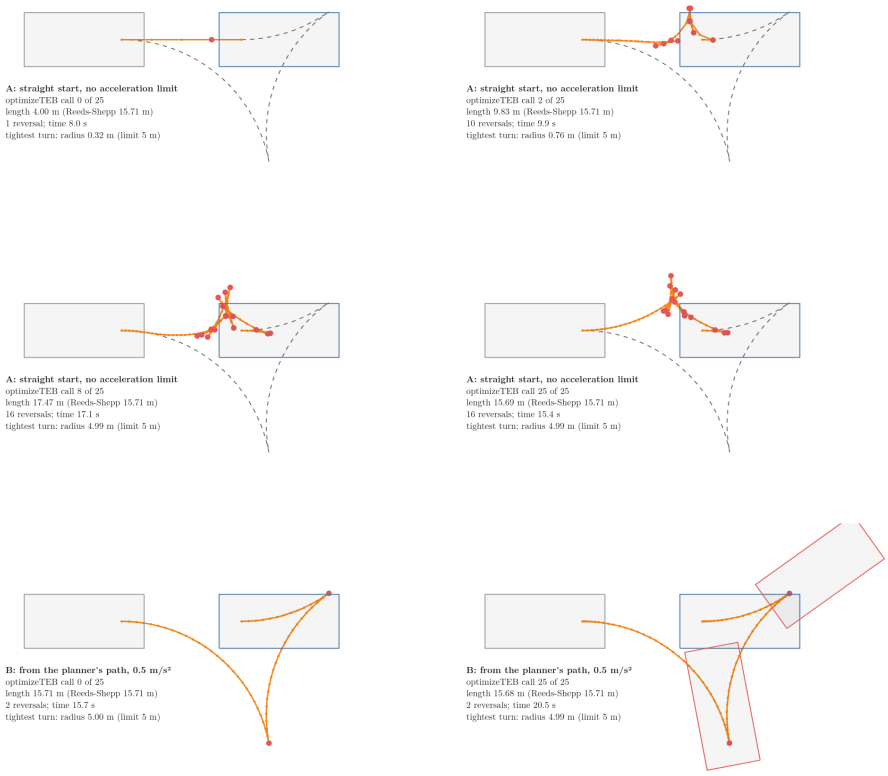


Figure 15: RO-055's car (wheelbase 2.5 m, tightest circle 5 m) turns round on an open floor, from (2, 0) facing 0° (blue outline) to (−2, 0) facing 180° (grey outline), with our Python version of TEB's algorithm, one frame per optimizeTEB call (4 passes of 5 steps). Grey dashed: the shortest Reeds–Shepp path, 15.708 m, forward left, reverse right, forward left. Orange: the band; red dots: where it changes direction. A: the paper's test, a straight start and no acceleration limit; the band grows to 15.69 m with 16 reversals. B: the band starts from the Reeds–Shepp path, as a car's global planner would give, with an acceleration limit of 0.5 m/s^2; it keeps 2 reversals, 15.68 m, and takes 20.5 s; the car is drawn at its two reversals

In test A the band grows from 4.00 m to 15.69 m, against 15.71 m for the Reeds–Shepp path, with its tightest turn at 4.99 m (notebook). The band reaches that length by many short arcs forward and back near the middle, 16 reversals, instead of the Reeds–Shepp path's 2. Both are equally short, because every arc of the tightest circle turns the car by its length divided by 5 m, so any path made only of such arcs that turns the car by π is equally long. With:

- ρ, the tightest turning radius, 5 m;
- $\Delta\theta_i$, the turn of arc i, all in the same direction.

$$5 \times \pi = 15.708 \text{ m}$$

$$\sum_i \rho \, |\Delta\theta_i| = \rho \, \pi$$

With no acceleration limit, a reversal costs no time, so the optimiser has no reason to prefer few of them. The paper found the same kind of result: in 8 of 12 goal poses its bands matched the Reeds–Shepp path, and the other 4 were local minima "of equal length and transition time" (Rösmann et al. 2017). At the paper's four smallest radii our lengths agree with its TEB lengths within 0.03 m; at the two largest they differ by 0.36 m and 0.91 m:

Radius	Ours	Paper's TEB	Reeds–Shepp (ours)
0.75 m	4.836 m	4.86 m	4.856 m
1.75 m	5.982 m	5.98 m	5.998 m
3.00 m	9.397 m	9.42 m	9.425 m
4.25 m	13.332 m	13.35 m	13.352 m
6.75 m	21.189 m	21.55 m	21.206 m
8.00 m	25.867 m	24.96 m	25.133 m

At 8 m our band stops 0.73 m longer than the Reeds–Shepp path, a local minimum (G-1110) (notebook).

In test B the band starts from the Reeds–Shepp path, the kind of first guess a car's global planner such as Hybrid A* gives, and the car may change speed by at most 0.5 m/s^2. Now each reversal costs a stop and a start, and the band keeps its 2 reversals: a three-point turn, 15.68 m in 20.5 s, tightest turn 4.99 m (notebook). One question remains: how TEB compares with the other kinds of local planner.

12. DWA, TEB and MPC compared

> **Key point:** In the authors' benchmark, TEB needs the least computing per step, 5.9 ms against 19.1 ms for the dynamic window's rollout and 12.5 to 15.8 ms for full model predictive control, and drives about as fast; the authors conclude that for a differential drive TEB need not be replaced.

The opening's cycle must fit in 0.1 s on a real computer, and the room is one test; a benchmark answers both for three families of local planner.

TEB's authors ran one differential-drive robot (speed −0.2 to 0.4 m/s, turn rate at most 0.4 rad/s) to three goals in a 10 m by 10 m map in ROS, with the global plan refreshed every 2 s and every local planner at 10 Hz (Rösmann et al. 2021, §IV-B). Figure 16 shows their Table I.

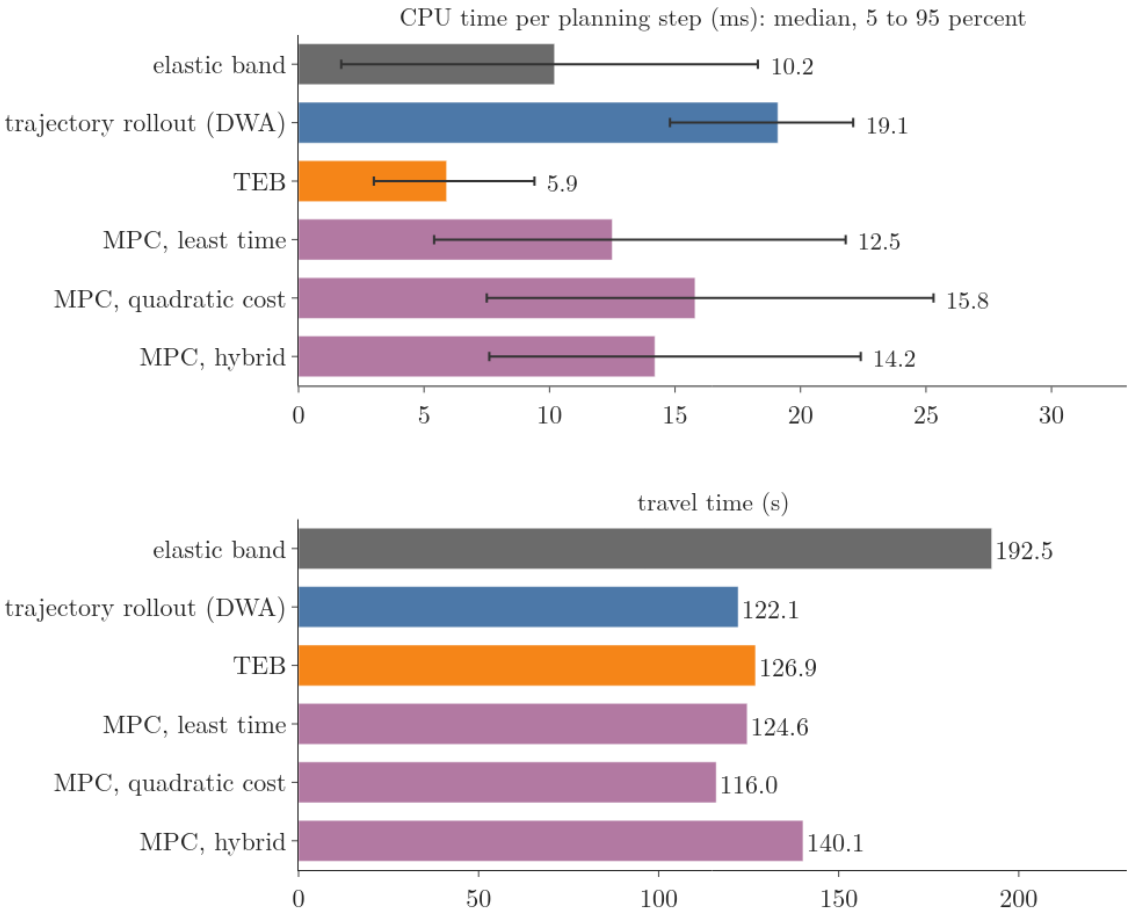


Figure 16: Rösmann et al. 2021, Table I: one differential-drive robot driving to three goals in a 10 m by 10 m map in ROS, local planners at 10 Hz. Top: computing time per planning step, median (bar) with the 5 and 95 percent quantiles (whiskers). Bottom: total travel time. Trajectory rollout is the DWA package in its rollout mode; the three MPC rows are the authors' model predictive controller with a least-time cost, a quadratic cost and a mix. Values typed in from the paper

- **The dynamic window** (DWA and DWB) holds one speed and turn rate over its whole horizon; the authors call it "a special receding horizon MPC scheme" whose search space is "a (collision-free) constant control input for the complete horizon", which "cannot predict motion reversals" (Rösmann et al. 2021, §I). The rollout took 19.1 ms per step and 122.1 s.
- **TEB** chooses every pose and time step: 5.9 ms per step and 126.9 s.
- **Model predictive control** (MPC), solving an optimal control problem again at every step from the measured state and applying only its first part, as RO-058's receding-horizon loop does, here with a general solver: 12.5 to 15.8 ms per step and 116.0 to 140.1 s.

Their conclusion: for simple robots such as a differential drive "there are no significant advantages … efficient planners such as the TEB approach need not be replaced"; full MPC

is worthwhile when the robot's model or the environment is more complex (Rösmann et al. 2021, §V). The faster travel times of rollout and quadratic MPC come, the authors write, from cutting corners because they have no fixed end pose. In our room the same ranking holds for the drive: TEB passes the person without a new global plan and satisfies Nav2's default test at 8.3 s, while DWB needs the plan remade and takes 13.3 s (Section 10).

13. Summary

Step	What the code does	In the room
inputs	plan, local costmap, pose and speed; defaults from `teb_config.h`	3.698 m plan, 3 m costmap, 0.5 m/s and 1.0 rad/s
plan ahead	prune, cut at 0.85 × half the costmap	32 poses, local goal 1.29 m ahead
obstacles	one point obstacle per lethal cell	590 cells, 534 points, 28 obstacle edges
first guesses	warm start; start again on a 90° goal turn; band from the plan	restart at 4.7 s, a 125.54° turn
candidates	one band per homotopy class, H-signature, random roadmap	left and right of the person at 2.5 s
optimise	g2o graph, 4 passes of 5 steps per band	89 numbers, 121 edges
select	cost with obstacles × 100, plan's class × 0.95	left band, 59.023 against 314.226
check, command	first 6 poses, first piece, limited	0.414 m clear; 0.3552 m/s
car	turning-radius edge, reversals, steering angle	15.69 m against 15.71 m; 26.58°

- In the cycle at 2.4 s our Python version of TEB's algorithm turned the 3.698 m plan and 590 lethal cells into 0.3552 m/s and 0.0238 rad/s; over the whole drive it passes the person 0.137 m clear and stops on the door pose at 9.8 s, because each cycle it bends a band round the person's cells without waiting for a new global plan, so it satisfies Nav2's default goal checker at 8.3 s, against 13.3 s for DWB, which must wait for a new plan.
- The plan is cut at 1.275 m in a 3 m costmap, so the **local goal** (G-2929) sits about 1.3 m ahead whatever the lookahead parameter says above that.
- Each **lethal cell** (G-3290) becomes a **point obstacle** (G-3291), so computing time grows with the costmap's area; a **costmap converter** (G-3292) groups cells into polygons to cut it.
- A **footprint model** (G-3293) sets how distance to the robot is measured, so a car needs two circles or a polygon where a disk robot needs a circle.
- A **warm start** (G-2914) is dropped when the local goal turns 90° or more, because a band shaped for one goal is a poor guess for another: at 4.7 s the goal turned 125.54°.
- **Homotopy-class planning** (G-3294) keeps one band per side of each obstacle, told apart by the **H-signature** (G-3296), because one band cannot cross an obstacle: with the person 0.25 m further left, TEB chose the left side where RO-058's single band overlapped the person.

- Every cost term is an **edge** (G-3298) of g2o's graph joining a few poses and time steps, so the problem is sparse and each band solves in a few milliseconds, 5.9 ms per step in the authors' benchmark.
- The **feasibility check** (G-3297) tests only the first 6 poses, because only the first piece is driven before the next cycle; a failure clears the bands and passes recovery to Nav2's behaviour tree.
- **Via-points** (G-3295) pull the band toward the plan, so the robot stays closer to it but arrives later: within 0.02 m at 10.4 s with weight 50, against 7.9 s without.
- A car needs `min_turning_radius`, which swaps in a turning-radius edge and allows reversals, so a three-point turn comes out of the optimisation: 15.68 m in 20.5 s from a planner's path.

The robot can now run a real local planner's algorithm step by step and see why it picks a side, slows, or refuses. One limitation remains: what decides to replan the path, clear the costmap or back up when TEB reports a failure, and how a goal is judged reached; behaviour trees and recovery (RO-079) answer that, and Nav2 start to finish (RO-080) puts every server together.

14. Sources

Built from

- teb_local_planner, GitHub, https://github.com/rst-tu-dortmund/teb_local_planner, branch `ros2-master` (commit e4562a62, November 2024), read 2026-10-12: `src/teb_local_planner_ros.cpp` (the cycle, plan pruning and cutting, obstacles from the costmap, backup modes, saturation, steering angle), `src/homotopy_class_planner.cpp` and `src/graph_search.cpp` (candidates, roadmap, depth-first search, selection, detours), `include/teb_local_planner/h_signature.h` (H-signatures), `src/optimal_planner.cpp` (graph, edges, passes, cost, command, feasibility check), `src/timed_elastic_band.cpp` (initialisation, warm start, resizing), `include/teb_local_planner/g2o_types/` (edges and penalties), `include/teb_local_planner/rob` `src/recovery_behaviors.cpp`, `include/teb_local_planner/teb_config.h` (defaults), `params/teb_params.yaml` (the Nav2 example file); branch `noetic-devel`: `cfg/TebLocalPlannerReconfigure.cfg` and `src/timed_elastic_band.cpp` (ROS 1 defaults and resizing) (teb_local_planner)
- teb_local_planner tutorials and FAQ, ROS wiki, read in Internet Archive copies: "Setup and test Optimization", "Following the Global Plan (Via-Points)", "Planning for car-like robots", "Obstacle Avoidance and Robot Footprint Model", "Configure and run Robot Navigation", FAQ, http://wiki.ros.org/teb_local_planner/Tutorials (TEB tutorial; TEB FAQ)
- Rösmann, C., Feiten, W., Wösch, T., Hoffmann, F. and Bertram, T. (2013). "Efficient trajectory optimization using a sparse model". *European Conference on Mobile Robots (ECMR)*, free PDF (Rösmann et al. 2013)
- Rösmann, C., Hoffmann, F. and Bertram, T. (2015). "Planning of multiple robot trajectories in distinctive topologies". *ECMR*, free PDF (Rösmann et al. 2015)
- Rösmann, C., Hoffmann, F. and Bertram, T. (2017). "Kinodynamic trajectory opti-

mization and control for car-like robots". *IEEE/RSJ IROS*, §II–III and Table I, free PDF (Rösmann et al. 2017)
- Rösmann, C., Makarow, A. and Bertram, T. (2021). "Online motion planning based on nonlinear model predictive control with non-Euclidean rotation groups". *European Control Conference*, arXiv 2006.03534: §I, §IV-B, Table I, §V (Rösmann et al. 2021)
- costmap_converter, GitHub, https://github.com/rst-tu-dortmund/costmap_converter, branch `ros2`: `costmap_converter/src/costmap_to_polygons.cpp` (costmap_converter)
- g2o, GitHub, https://github.com/RainerKuemmerle/g2o, `g2o/core/optimization_algorithm_leve` (the step-size rule) (g2o)
- Nav2 documentation, https://docs.nav2.org, "Navigation Plugins" (TEB as a third-party controller) (Nav2 docs); navigation2, GitHub, https://github.com/ros-navigation/navigation2, branch `main`, read 2026-10-12: `nav2_bringup/params/nav2_params.yaml` (local costmap, controller frequency, failure tolerance, goal checker) (Nav2 `nav2_params.yaml`), `nav2_costmap_2d/include/nav2_costmap_2d/cost_values.hpp` (lethal cost 254) (navigation2, `cost_values.hpp`)

Other references

- teb_local_planner, GitHub issues 195 ("Stopping accuracy is not met") and 406 ("teb_local_planner drifts near the goal"), https://github.com/rst-tu-dortmund/teb_local_planner/issues (teb_local_planner, issues 195 and 406)
- Kümmerle, R., Grisetti, G., Strasdat, H., Konolige, K. and Burgard, W. (2011). "g2o: a general framework for graph optimization". *IEEE ICRA*
- Macenski, S., Moore, T., Lu, D. V., Merzlyakov, A. and Ferguson, M. (2023). "From the desks of ROS maintainers: a survey of modern and capable mobile robotics algorithms in the Robot Operating System 2". *Robotics and Autonomous Systems*, arXiv 2307.15236, §III: TEB among Nav2's controllers

15. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Via-point (G-3295)	A point a local planner's trajectory is pulled towards by a weighted cost term, sent by another program or taken from the global plan at a fixed spacing; it keeps the robot closer to the plan at the price of time.
Lethal cell (G-3290)	A costmap cell that holds an obstacle, with the highest cost (254 in Nav2), so that a robot centred on it would certainly collide; local planners such as TEB turn these cells into the obstacles they avoid.

Term	Meaning
Point obstacle (G-3291)	An obstacle given as a single point, such as the centre of one lethal cell; TEB by default makes one per lethal cell of its local costmap and keeps the robot's footprint a set distance from each, so its computing time grows with the number of cells.
Costmap converter (G-3292)	A plugin that groups neighbouring lethal cells of a costmap into a few polygons or lines, such as a convex polygon round each group of cells within 0.4 m, and hands those to TEB in place of one point per cell, so each cycle has fewer obstacles to check.
Footprint model (G-3293)	The shape a local planner uses for the robot when it measures the distance to an obstacle: in TEB a point, a circle, two circles, a line or a polygon; a coarser shape is faster to compute, a closer one wastes less room, such as two circles or a polygon for a car.
Homotopy-class planning (planning in distinctive topologies) (G-3294)	Keeping several trajectories that pass obstacles on different sides, one per homotopy class, optimising them all and choosing the cheapest; a local optimiser cannot move one trajectory across an obstacle, so this lets it switch sides, as TEB does with up to 5 bands.
H-signature (G-3296)	A number computed for each obstacle from the angle a path turns through as seen from that obstacle, positive when the obstacle lies on the path's left; paths whose numbers have the same signs are in the same homotopy class, so TEB uses it to tell its bands apart.
Edge (g2o graph) (G-3298)	One cost term of a graph optimisation, joined to the vertices whose values it reads, such as TEB's acceleration term joined to three poses and two time steps; because each edge touches only a few vertices, the problem is sparse and fast to solve.
Feasibility check (G-3297)	Testing the first few poses of a planned trajectory against the costmap's lethal cells before it is used, and rejecting it if the robot's footprint touches one; in TEB the robot and the next 5 poses, because only the first piece is driven before the next plan.

Term	Meaning
Local planner (G-2911)	The planner that turns the next few metres or seconds of the global plan into a timed trajectory within the robot's limits and around obstacles its sensors see, solving again many times a second, such as TEB at 10 Hz over 1.5 m; it handles what the global plan cannot.
Timed elastic band (TEB) (G-2920)	A trajectory written as a list of poses with the time between neighbours, optimised for the least total time while penalties keep the motion rule, the speed and acceleration limits and the clearance from obstacles; a ROS and Nav2 local planner that solves it again every cycle from its last answer.
Occupancy grid (G-2340)	A grid map that stores in each cell a number between 0 and 1, such as the probability that the cell is occupied, instead of a hard 0 or 1; the standard grid map of robotics, since sensor readings are uncertain.
Local goal (virtual goal) (G-2929)	The pose on the global plan at the end of the local planner's window, facing along the plan, which the local planner plans to reach; it moves along the plan with the robot, and becomes the real goal when that comes within reach.
Warm start (G-2914)	Starting a solver from an earlier answer, such as the last cycle's trajectory or a planner's path, instead of from scratch; each solve then only corrects a nearby answer, and the starting point also decides which local minimum it reaches.
Homotopy class (G-2922)	A set of paths with the same start and goal that can be bent into one another without crossing an obstacle, such as all paths passing a person on the left; an optimiser never leaves the class it starts in, so a search must pick the class.

Term	Meaning
Depth-first search (DFS) (G-2552)	A search whose frontier is a stack, so it always extends the newest path and dives deep before backing up; it backs up to its last untried choice at dead ends and finds a route, but which one, and how long, depends on the order the moves are tried (22 moves where breadth-first search found 8).
Probabilistic roadmap (PRM) (G-2780)	A graph of random free configurations (milestones) joined by straight edges that pass a collision check, built once and then searched with A*; it describes the free space without building the obstacle region, so it scales to robots with many joints.
Levenberg–Marquardt method (G-2921)	A solver for sums of squares that takes Gauss–Newton steps, jumping to the lowest point of a bowl built from straight-line guesses of each term, blended with a small step downhill that grows when a step fails and shrinks when it succeeds; it converges fast near the answer and safely far from it.
g2o (G-2927)	A library that solves sparse least-squares problems written as a graph, each term joining a few variables, with Gauss–Newton or Levenberg–Marquardt steps; TEB solves its band with it.
Turning radius R (G-2293)	The distance from the point a robot is turning about (the instantaneous centre of curvature, ICC) to the robot's reference point; $R = v/\omega$, so it says how tight a turn is.
Reeds-Shepp path (Reeds-Shepp curve) (G-2829)	The shortest path between two poses for a vehicle that turns no tighter than a given radius but may also reverse: at most five full-lock arcs and straight pieces with at most two reversals, from 48 words in nine families; planners use it to join two poses exactly with a drivable path.
Local minimum (G-1110)	A point lower than everything around it, but not the lowest overall.